

REPUBLIC OF CAMEROON

Peace - Work - Fatherland



**AFRICAN INSTITUTE OF COMPUTER
SCIENCES - CAMEROON OFFICE
PAUL BIYA TECHNOLOGICAL CENTER OF
EXCELLENCE**

P.O. Box: 13719 Yaoundé
Tel: +237.242.729.957 Fax: 22729958
E-mail: contact@iaicameroun.com
Web site: www.iaicameroun.com

REPUBLIQUE DU CAMEROUN

Paix – Travail - Patrie



REALIZE

Tel: (237) 620256960/654486985
Email : realize.contactus@gmail.com
Web site: www.realize.cm

THEME: DESIGN AND IMPLEMENTATION OF AN AI- ASSISTED PHARMACY STOCK MANAGEMENT PLATFORM WITH GEOLOCATION-BASED DRUG SEARCH AND DELIVERY SERVICE

Internship carried out from the 01st of July till the 30th of September 2026. In view
of obtaining a Software Engineering Diploma in Computer Sciences Option:
Software Engineering

Written by:

NSAN SAMUEL PENIEL

Level III Software Engineering student

Academic Supervisor:

Mr. ASANE DERICK

Lecturer at AICS-Cameroon

Professional Supervisor:

Mr. MENGOT NSO

Software Engineer

Academic Year 2025 – 2026



DESIGN AND IMPLEMENTATION OF AN AI-ASSISTED PHARMACY
STOCK MANAGEMENT PLATFORM WITH GEOLOCATION-BASED
DRUG SEARCH AND DELIVERY SERVICE



DEDICATION

TO THE DIBANGO
FAMILY

Written By: NSAN SAMUEL PENIEL

I



ACKNOWLEDGEMENTS

As a prelude to this internship report, we would like to express our sincere gratitude to all individuals and entities whose guidance, support, and encouragement have contributed to the successful completion of this work and to a productive academic year. We are particularly grateful to:

- ✚ **Mr. ARMAND CLAUDE ABANDA**, the Resident Representative of the African Institute of Computer Sciences, for his wise and invaluable instructive advice.
- ✚ **Mr. MENGOT NSO**, our professional supervisor, for his help, advice, and encouragement that greatly supported the successful development of this project.
- ✚ **Mr. ASANE DERICK**, our academic supervisor and lecturer at AICS Cameroon, for his constant guidance, advice, and moral support during our academic and internship journey, which contributed to the realization of this project.
- ✚ The **Administrative and Teaching Staff of AICS Cameroon**, for their dedication, teaching, and guidance throughout the 2023-2026 academic year.
- ✚ Our **families and parents**, for their spiritual, moral, and financial support, which was instrumental in enabling us to complete this work.
- ✚ Lastly, all other individuals and organizations who contributed in one way or another to the realization of this project.



DESIGN AND IMPLEMENTATION OF AN AI-ASSISTED PHARMACY
STOCK MANAGEMENT PLATFORM WITH GEOLOCATION-BASED
DRUG SEARCH AND DELIVERY SERVICE



CONTENT

Written By: NSAN SAMUEL PENIEL

III



DESIGN AND IMPLEMENTATION OF AN AI-ASSISTED PHARMACY
STOCK MANAGEMENT PLATFORM WITH GEOLOCATION-BASED
DRUG SEARCH AND DELIVERY SERVICE



LIST OF FIGURES

Written By: NSAN SAMUEL PENIEL

IV



DESIGN AND IMPLEMENTATION OF AN AI-ASSISTED PHARMACY
STOCK MANAGEMENT PLATFORM WITH GEOLOCATION-BASED
DRUG SEARCH AND DELIVERY SERVICE



LIST OF TABLES



LIST OF ABBREVIATIONS

- **AICS:** African Institute of Computer Sciences
- **AI :** Artificial Intelligence
- **API :** Application Programming Interface
- **CD-ROM:** Compact Disc Read-Only Memory
- **CSS :** Cascading Style Sheets
- **HTML :** Hypertext Markup Language
- **IAI :** Institut Africain de L'Informatique
- **ICT :** Information and Communication Technologies
- **MERISE :** Méthode d'Étude et de Réalisation Informatique pour les Systèmes d'Entreprise
- **MOMO :** Mobile Money
- **MTN :** Mobile Telecommunications Network
- **OM :** Orange Money
- **2TUP :** Two-Track Unified Process
- **TIC :** Technologies de l'Information et de la Communication
- **UI :** User Interface
- **UML :** Unified Modeling Language.
- **HYSACAM :** Hygiene and Sanitation of Cameroon



ABSTRACT

The rapid evolution of Information and Communication Technologies (ICT) has transformed several sectors, including healthcare, by enabling innovative solutions that improve access to essential medical services. In Cameroon, clients may face difficulties in identifying nearby pharmacies with the required medicines, verifying their actual availability, and arranging convenient pickup or delivery. In this context, digital platforms can improve pharmacy services by facilitating stock management, medicine search, order processing, and delivery. The African Institute of Computer Sciences Paul Biya Technological Centre of Excellence (IAI-Cameroon) promotes practical learning by combining academic training with hands-on project experience through internships. During my academic internship, I contributed to the development of **PharmaGo**, based on the theme “**Design and Implementation of an AI-Assisted Pharmacy Stock Management Platform with Geolocation-Based Drug Search and Delivery Service.**” The project involved feasibility studies, requirements analysis, UML modeling, design, implementation, and testing using the **2TUP process**. PharmaGo provides pharmacy stock management, geolocation-based drug search, pharmacy location, order processing, prescription management, pickup and delivery services, and Mobile Money payment. The platform consists of a Flutter mobile application for clients, a web portal for pharmacy and administrative users, an Express.js backend, and a PostgreSQL database. Google Maps is integrated to support pharmacy location, nearby pharmacy search, routes, and estimated travel information. An AI assistant powered by the Gemini API provides advisory symptom-checking support while encouraging users to consult qualified healthcare professionals when necessary. Through these functionalities, PharmaGo aims to improve medicine availability, facilitate interaction between clients and pharmacies, and provide a more efficient and accessible pharmacy service in Cameroon.

Keywords: PharmaGo; Pharmacy Stock Management; Artificial Intelligence; Geolocation; Drug Search; Medicine Delivery; Flutter; 2TUP; Gemini API; Cameroon.



RESUME

L'évolution rapide des Technologies de l'Information et de la Communication (TIC) a transformé plusieurs secteurs, notamment celui de la santé, en permettant le développement de solutions innovantes destinées à améliorer l'accès aux services médicaux essentiels. Au Cameroun, les clients peuvent rencontrer des difficultés pour identifier les pharmacies proches disposant des médicaments recherchés, vérifier leur disponibilité réelle et organiser leur retrait ou leur livraison. Dans ce contexte, les plateformes numériques peuvent contribuer à améliorer les services pharmaceutiques en facilitant la gestion des stocks, la recherche de médicaments, le traitement des commandes et la livraison. L'Institut Africain d'Informatique, Centre d'Excellence Technologique Paul Biya (IAI-Cameroun), favorise l'apprentissage pratique en associant la formation académique à l'expérience acquise au cours des stages. Durant mon stage académique, j'ai contribué au développement de **PharmaGo**, réalisé suivant le thème « **Conception et mise en œuvre d'une plateforme de gestion des stocks pharmaceutiques assistée par l'intelligence artificielle, avec recherche géolocalisée de médicaments et service de livraison** ». Le projet a porté sur l'étude de faisabilité, l'analyse des besoins, la modélisation UML, la conception, l'implémentation et les tests suivant le **processus 2TUP**. PharmaGo intègre la gestion des stocks pharmaceutiques, la recherche géolocalisée de médicaments, la localisation des pharmacies, la gestion des commandes, la gestion des prescriptions, ainsi que les services de retrait en pharmacie et de livraison. La plateforme prend également en charge le paiement par Mobile Money. Elle repose sur une application mobile développée avec Flutter pour les clients, un portail web destiné aux utilisateurs pharmaceutiques et administratifs, un backend basé sur Express.js et une base de données PostgreSQL. Google Maps est intégré pour la localisation des pharmacies, la recherche des pharmacies proches,.

Mots-clés : PharmaGo ; Gestion des stocks pharmaceutiques ; Intelligence artificielle ; Géolocalisation ; Recherche de médicaments ; Livraison de médicaments ; Flutter ; 2TUP ; API Gemini ; Cameroun.



GENERAL INTRODUCTION

In today's fast-paced world, technological advancements are transforming various sectors, including waste management. Digital innovations are helping cities and enterprises improve efficiency, reduce costs, and address environmental challenges. As a developing country, Cameroon is increasingly embracing the digital economy to boost growth and sustainability. To contribute to this progress, students at AICS Cameroon are required to undertake a three-month internship to apply their academic knowledge in real-world scenarios. During our internship at **REALIZE**, an IT solutions company, we were assigned the theme: **“Conception and Implementation of a Waste Collection System.”** Our project aims to design and develop a digital platform that optimizes waste collection processes, enhances efficiency, and promotes better environmental management. To achieve this, we conducted comprehensive research and structured our work into eight parts:

1. **Insertion Document:** Introducing the company and our internship experience, along with the project theme.
2. **Existing System:** Analyzing current waste collection methods.
3. **Specification Book:** Defining user needs, considering costs and deadlines.
4. **Analysis Document:** Presenting analysis methods and diagrams for understanding the system.
5. **Conception Phase:** Designing the overall and detailed architecture of the waste collection system.
6. **Realization Phase:** Demonstrating how the system will be built and implemented.
7. **Functionality Testing:** Highlighting the modules and how they operate.
8. **User Guide:** Providing instructions for users to operate the system effectively.

Written By: NSAN SAMUEL PENIEL



DESIGN AND IMPLEMENTATION OF AN AI-ASSISTED PHARMACY
STOCK MANAGEMENT PLATFORM WITH GEOLOCATION-BASED
DRUG SEARCH AND DELIVERY SERVICE



PART 1 : INSERTION PHASE

Written By: NSAN SAMUEL PENIEL

2



Preamble

This report summarizes the company, the project we worked on, the tasks we completed, the tools used, and the results achieved during the internship.

Content Overview

INTRODUCTION

- I. WELCOME AND INTEGRATION
 - II. GENERAL PRESENTATION OF REALIZE
 - III. ORGANISATION OF REALIZE
 - IV. HARDWARE AND SOFTWARE RESOURCES
 - V. BRIEF PRESENTATION OF THE PROJECT
- ### CONCLUSION



INTRODUCTION

Insertion is the way in which man can integrate a community, a group. Within the framework of the academic internship, the insertion phase is a period in which the different interns have to know more about their host company. Additionally, we have to know about the staffs, the different hardware and software used, the different department, which constitute the company, and we were introduced to our workspace, how the company functions both internally (that is how the different task is schedule, the rules and regulations, working periods and so on). We also had a time to discuss amongst us interns on topics like what we love doing most, what we dislike, our believes, our best meals, sports, songs, our temperaments, our inborn talents and those we learn as we grow up. We shared about different realizations and failures in life.

Written By: NSAN SAMUEL PENIEL



I. WELCOME AND INTEGRATION

a. Welcome

We arrived at REALIZE on Thursday, July 1, 2026, at 08:00. Greeting us were **Mr. TANUE MONET**, the Software Designer, and **Mr. MENGOT NSO**, the Senior Developer, who delivered an inspiring welcome and introduced the company's operations and scheduling for our internship period. The initial meeting covered the challenges from the coding test required for the internship, REALIZE's organizational structure and internal policies, and the allocation of workstations to each intern.

b. Integration

First-week activities: We set up at our designated workstations and received our initial task. A foundational verification of concepts using HTML and CSS ensured all interns started on an even footing. We brainstormed project ideas, with an emphasis on individual initiative; prior experience helped me contribute and refine these ideas. We conducted research on the proposed ideas, as REALIZE encouraged initiative and did not prescribe themes.

Second-week activities: JavaScript concept verification exercises were used to assess understanding and reasoning. A detailed discussion with the supervisor reviewed the specification book and its components, clarifying expectations for each section. Project ideas were further developed into concrete plans, leveraging familiarity with REALIZE's expectations and processes.

Written By: NSAN SAMUEL PENIEL



II. GENERAL PRESENTATION OF REALIZE

A. History

Realize is a Cameroon-based NGO and tech start-up founded by **Mr. NDELOGAKEH Daniel** in 2022. It offers IT solutions and promotes the use of new technologies in Cameroon.

B. What is REALIZE all about?

i. Missions

Realize mission is to empower and sustain the use of technology through:

- Designing mobile software for businesses and individuals
- Supporting startups for rapid growth using IT
- Providing training and certifications to build qualified human resources
- Contributing to global sustainable development with innovative solutions and virtual reality

ii. Vision

Realize believes that unlocking the potential of organizations and individuals is possible through analysis, development, realization, and data analytics. Their vision is to shape a future of limitless innovation in the digital world.

iii. Activities

Realize activities include:

- Designing and hosting websites
- Software development and maintenance
- Providing training in software fields
- Creating multimedia content
- Offering IT consulting, support, and innovation.



C. GEOGRAPHICAL LOCATION

REALIZE is located at Yaoundé-Valle Amadou



Figure 1: Realize Location

Written By: NSAN SAMUEL PENIEL



III. ORGANISATION OF REALIZE

A. Administrative Organization

Realize's administrative structure is divided into several departments:

1. General Management

- Oversees the entire company
- Ensures departments function properly
- Sets project strategies
- Provides leadership and guidance
- Makes critical decisions impacting operations and reputation
- Acts as a point of contact for stakeholders like investors

2. Human Resources

- Handles recruitment and hiring
- Acts as a link between employees and management
- Manages salaries, benefits, and employee policies
- Maintains employee records and HR systems

3. Communication Department

- Manages public relations and company image
- Handles messaging to external stakeholders (customers, partners)
- Ensures effective internal communication
- Creates and manages content for various platforms

4. Financial Affairs

- Develops and manages budgets and financial plans
- Prepares financial reports for management and authorities
- Controls costs and expenses
- Ensures tax compliance

Written By: NSAN SAMUEL PENIEL



5. Technical Department

- Manages infrastructure like networks and hardware
- Handles data management and cybersecurity
- Maintains relationships with vendors and service providers

6. Software Engineering Department

- Develops and supports software applications
- Maintains software throughout its lifecycle
- Executes and evaluates project realization

B. FUNCTIONAL ORGANISATION OF REALIZE

The functional branch of realize is organized as follows:

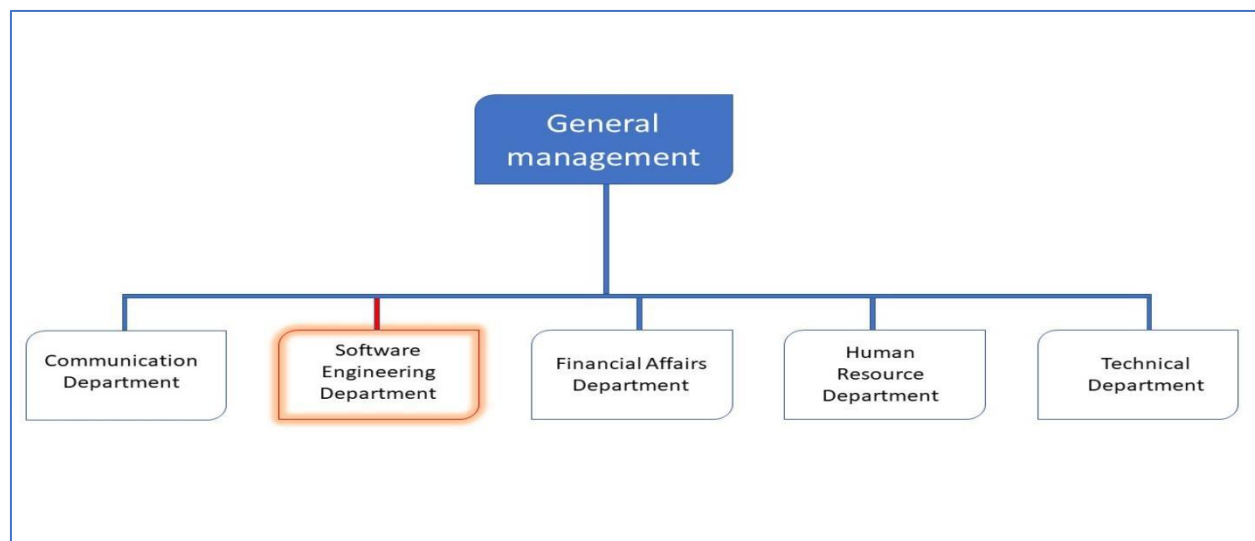


Figure 1: Company functional organization (Source: Realize)



IV. HARDWARE AND SOFTWARE RESOURCES OF REALIZE

A. HARDWARE RESOURCES

Table 1 : Hardware resources (Source from REALIZE)

No	DESIGNATION	QUANTITY	CHARACTERISTICS	OBSERVATION
1	MacBook Pro	1	APPLE	Good
2	Modem	1	CAMTEL	Good
3	LCD Screen	1	HP	Excellent
4	Training equipment and office furniture	/	/	/

B. SOFTWARE RESOURCES

Table 2 : Software resources (Source REALIZE)

DESIGNATION	SOFTWARE
Operation system	Windows 10, Mac OS, parrot Linux
Design tools	Photoshop
Integrated development environment (IDE)	Visual studio code,
Text editor	Sublime text, notepad++
Database management system (DBMS)	MongoDB
Web browser	Google chrome, Microsoft edge
Document editor	Microsoft office word
Presentation	Microsoft office PowerPoint

V. BRIEF PRESENTATION OF THE PROJECT

Written By: NSAN SAMUEL PENIEL



DESIGN AND IMPLEMENTATION OF AN AI-ASSISTED PHARMACY STOCK MANAGEMENT PLATFORM WITH GEOLOCATION-BASED DRUG SEARCH AND DELIVERY SERVICE



During our insertion phase at REALIZE, we identified themes addressing specific problems and introducing innovation. After reviewing our initial ideas, our professional supervisor took the time to evaluate our themes and proposed some improvements. Finally, we had to go for the selected theme: **"CONCEPTION AND REALIZATION OF A WASTE COLLECTION SYSTEM"**. This platform aims to improve waste collection processes, making them more efficient and organized. It will help waste collectors and community members by providing a digital solution to manage their waste collection activities. Each contributor (community member or household) will have the ability to view their assigned collector, see the history of waste collected, and make online requests for waste pick-up. Collectors will be able to register waste contributions through the application and issue receipts to contributors for each collection. Additionally, the admin will have the capability to assign collectors to specific contributors, thereby streamlining the entire waste management process and reducing environmental nuisances caused by uncollected waste.

Written By: NSAN SAMUEL PENIEL



CONCLUSION

The insertion phase at REALIZE established the foundation for our internship, enabling rapid integration into the Software Engineering host unit and a clear understanding of the organization's structure, workflows, and resources. We completed initial HTML/CSS exercises to validate core concepts and began shaping project ideas, preparing us to transition from onboarding to hands-on work. The phase culminated in the shift toward conceiving and implementing a waste collection system, setting the stage for practical design, stakeholder engagement, and scalable, impact-driven development.

Written By: NSAN SAMUEL PENIEL

12



PART 2 : TECHNICAL PHASE



DESIGN AND IMPLEMENTATION OF AN AI-ASSISTED PHARMACY
STOCK MANAGEMENT PLATFORM WITH GEOLOCATION-BASED
DRUG SEARCH AND DELIVERY SERVICE



FILE 1: EXISTING SYSTEM

Written By: NSAN SAMUEL PENIEL

14



Preamble

This section explains the existing system we are studying and outlines the app we will build. It describes how the existing system works, where it falls short, and the problems this causes. It then presents practical fixes and the initial steps to implement them.

Content Overview

INTRODUCTION

- I. THEME PRESENTATION
- II. DESCRIPTION OF THE EXISTING SYSTEM
- III. LIMITATIONS OF THE EXISTING SYSTEM
- IV. PROBLEMATIC
- V. PROPOSED SOLUTION

CONCLUSION



INTRODUCTION

In this part of our report, we will explain our theme. We will also study the current system and identify its strengths and weaknesses. Based on this analysis, we will present the problem statement, which is the main challenge or gap that the current system faces. Finally, we will propose a solution that addresses this problem and improves the system.



I. THEME PRESENTATION

Our theme, “**Design and Implementation of a Pharmacy Stock Management Platform with Geolocation-Based Drug Search and Delivery: Case of Cameroon**”, focuses on designing and developing a digital platform to improve how pharmaceutical stock is managed and how clients access medicines from nearby pharmacies. The system connects **clients, pharmacies, Pharmacy Staff, delivery agents, and the platform administrator** through an intuitive digital platform. It addresses important limitations in traditional pharmacy services, particularly the difficulty of knowing whether a specific medicine is available before travelling to a pharmacy, as well as the challenges associated with ordering and delivery. The conception phase involves analysing existing pharmacy stock management and medicine procurement practices, identifying limitations such as **inaccurate or non-real-time stock information, difficulty locating nearby pharmacies, limited order tracking, manual prescription verification, and difficulties accessing medicines through delivery services**, and proposing a digital solution adapted to the Cameroonian context. The implementation phase consists of developing an **Android-first mobile application for clients** and a **web-based dashboard for Pharmacy Staff and the Platform Admin**, supported by a common backend API. The platform enables clients to search for medicines, activate their GPS location, identify nearby pharmacies with the requested medicine listed in stock, view pharmacy information and route details, request orders, upload prescriptions when required, choose between pickup and delivery, make payments using Mobile Money or Cash on Delivery, and track their orders.



II. DESCRIPTION OF THE EXISTING SYSTEM

In Cameroon, access to medicines generally depends on clients physically visiting pharmacies or contacting them individually to determine whether a particular medicine is available. Pharmacy personnel are responsible for managing medicine stocks, receiving clients, verifying prescriptions when necessary, processing sales, and handling other pharmacy operations. In the traditional approach, **stock management is mainly handled internally by each pharmacy**. Pharmacy staff record and monitor available medicines and their quantities. However, clients generally do not have direct access to this information before visiting the pharmacy. Consequently, a client who needs a specific medicine may have to visit or contact several pharmacies before finding one that has it available. This situation becomes more problematic when the medicine is urgently required or when the client is located far from the pharmacy. Clients may spend significant amounts of time and transportation costs searching for medicines that may eventually be unavailable. Pharmacies also face operational challenges. Stock information may not always be updated immediately after a medicine is sold or received. This can result in differences between the quantity recorded in the pharmacy's stock and the quantity actually available. For this reason, simply displaying stock information to a client cannot always guarantee actual availability. The traditional system also provides limited support for **geolocation-based pharmacy discovery**. Clients may know the name of a pharmacy but still have difficulty determining which pharmacy is closest, whether it is currently open, or how long it would take to reach it. Furthermore, the process of ordering medicines remotely is not always centralized. Clients may need to call or communicate directly with pharmacies, while order confirmation, prescription verification, payment and delivery may be handled through separate manual processes.



III. LIMITATIONS OF THE EXISTING SYSTEM

Table 3 : Limitations, Consequences and proposed solutions

Limitations	Consequences	Proposed Solutions in PharmaGo
No centralized digital access to pharmacy stock information	Clients must contact or visit several pharmacies to find a medicine	A digital stock management platform allowing pharmacies to maintain their medicine stock and clients to search for available medicines
Stock information may be outdated or inaccurate	Clients may travel to a pharmacy only to discover that the medicine is unavailable	Pharmacy Staff manually confirm actual availability before an order is finalized
Difficulty locating nearby pharmacies	Clients spend additional time and transportation costs searching for medicines	GPS-based pharmacy discovery using Google Maps
No integrated route and travel information	Clients may have difficulty determining how to reach the selected pharmacy	Google Maps integration for location, route, ETA and travel time
Limited digital ordering	Clients may need to contact pharmacies manually to request medicines	Digital order request through the mobile application
Manual prescription management	Prescription verification can be slow and difficult to track	Digital prescription upload and verification by Pharmacy Staff
Limited order status visibility	Clients may not know whether an order has been accepted or prepared	Digital order status updates and notifications
Limited payment options within the platform	Payment may require separate manual processes	Integration with Mobile Money payment gateway and Cash on Delivery
Limited coordination of pharmacy deliveries	Delivery preparation and assignment may be difficult to monitor	Digital delivery assignment to the pharmacy's own delivery agents
No centralized history of orders	Clients and pharmacies have limited access to previous transactions	Digital storage of order and transaction history
Limited access to health information	Clients may lack basic health-related guidance	Health tips section within the platform
No integrated advisory symptom-checking tool	Clients may have difficulty deciding whether to seek professional medical advice	Advisory-only AI symptom checker that encourages consultation with a healthcare professional



IV. PROBLEMATIC

How can a digital pharmacy stock management platform improve the visibility and reliability of medicine availability while enabling clients to locate nearby pharmacies, request and confirm orders, and obtain medicines through pickup or pharmacy-based delivery?

V. PROPOSED SOLUTION

The proposed solution is **PharmaGo**, a digital pharmacy stock management platform designed to improve medicine availability visibility and facilitate access to medicines in Cameroon. The platform combines **pharmacy stock management, geolocation-based medicine search, digital ordering, prescription management, payment, and pharmacy-based delivery** within a unified system.

1. Pharmacy Stock Management and Geolocation of a pharmacy

The platform provides Pharmacy Staff with a web dashboard for managing pharmacy stock. The stock information is then used by the system when clients search for medicines.

2. Order Request and Human Confirmation

After finding a suitable pharmacy, the client selects the medicine and quantity and submits an order.

3. Delivery and Payment Management

Mobile Money processing can be demonstrated using a sandbox or simulated payment flow during the academic defense.

4. Order Tracking and Notifications

The platform allows clients to follow the progress of their orders and For delivery orders, the system can use route information to support delivery tracking.

5. Health Information and Advisory Symptom Checker

PharmaGo also provides a **Health Tips** section containing general health information managed by the Platform Admin.



CONCLUSION

In conclusion, the current waste collection system faces challenges in efficiency, accessibility, and payment management, particularly in hard-to-reach areas. The proposed digital solution, combining a mobile app, GPS tracking, online payments, and integrated management tools, offers a practical and effective way to optimize waste collection, ensure timely service, and improve overall transparency and customer satisfaction.



FILE 2: SPECIFICATION BOOK



Preamble

The primary goal of the specification book is to outline with great precision the requirements or need of the users and the description of the resources necessary to realize the project. The project's success will depend on careful allocation and management of the various resources available, including physical, financial, and human assets. Therefore, this document outlines the criteria that the developer must adhere to in order to ensure the success of the project.

Content Overview

INTRODUCTION

- I. CONTEXT AND JUSTIFICATION
 - II. OBJECTIVES OF THE PROJECT
 - III. EXPRESSION OF NEEDS
 - IV. PROJECT PLANNING
 - V. PROJECT ESTIMATION COST
 - VI. PROJECT CONSTRAINTS
 - VII. LIST OF PARTICIPANTS AND DELIVERABLES
- ### CONCLUSION



INTRODUCTION

The report's specification book enables us to provide comprehensive details about our topic, enhancing our understanding and improving its chances of success. To establish the project's scope, we will describe the context of our topic and identify the issues within it that we aim to solve. Once the solution is proposed, we will outline the objectives we have set for ourselves and for the project. It establishes a certain agreement between the client of the project and the person who is supposed to realize the project. It presents the objectives and needs of the system to be developed. The present specification book relates to the context and justification, the needs of the project, the plan and the deliverables.



I. CONTEXT AND JUSTIFICATION

1. Context

Cameroon has a growing population and an expanding urban environment, particularly in major cities such as **Yaoundé, Douala, and Bafoussam**. This growth is accompanied by increasing demand for accessible and reliable healthcare services, including timely access to medicines. Although pharmacies are widely distributed in urban areas, obtaining a specific medicine can still be difficult when clients do not know which nearby pharmacy has the required product in stock. In the traditional pharmacy environment, clients generally have to **visit or contact pharmacies individually** to determine whether a medicine is available. This process becomes particularly inconvenient when a medicine is urgently needed or when the client is located far from the pharmacy. Another important challenge concerns **pharmacy stock management**.

Key challenges include:

- **Limited stock visibility:** Clients generally cannot easily determine which nearby pharmacy has a particular medicine available.
- **Difficulty locating pharmacies:** Clients may spend time searching for pharmacies that are close to their current location.
- **Unnecessary travel:** A client may travel to a pharmacy only to discover that the required medicine is unavailable.
- **Outdated stock information:** Pharmacy stock records may not always reflect the actual physical quantity available.
- **Limited digital ordering:** Medicine requests are often handled through physical visits, telephone calls, or informal communication channels.
- **Prescription management difficulties:** Prescription verification may still depend on manual processes.



2. Justification

The justification for developing a computerized pharmacy stock management platform such as **PharmaGo** lies in addressing the limitations associated with traditional medicine-searching, stock-management, ordering, and delivery processes.

The digitalization of these processes will:

- **Improve pharmacy stock visibility** by allowing Pharmacy Staff to maintain updated information about medicines and quantities.
- **Reduce unnecessary trips** by enabling clients to search for a medicine and identify nearby pharmacies before travelling.
- **Improve the reliability of availability information** through the required human confirmation step, whereby Pharmacy Staff verify the actual availability of the requested medicine before the order is finalized.
- **Facilitate geolocation-based pharmacy discovery** by using Google Maps to identify nearby pharmacies and provide their location, open/closed status, route, ETA, and travel time.
- **Simplify medicine ordering** by allowing clients to submit orders directly through the mobile application.
- **Facilitate prescription management** by allowing clients to upload prescriptions and Pharmacy Staff to verify them digitally.
- **Improve access to medicines** by providing both pharmacy pickup and pharmacy-based delivery options.
- **Improve payment convenience** through Mobile Money and Cash on Delivery.
- **Increase transparency** by allowing clients to follow the status of their orders.



II. OBJECTIVES OF THE PROJECT

A. General Objective

To design and implement a digital pharmacy stock management platform that improves medicine availability visibility and facilitates the geolocation-based search, ordering, confirmation, ai symptom checker and delivery of medicines in Cameroon.

B. Specific Objectives

The specific objectives of this project are to:

- **Develop a pharmacy stock management module** allowing Pharmacy Staff to add, update, remove, and monitor medicine stock records.
- **Facilitate medicine search** by allowing clients to search for a specific medicine through the mobile application.
- **Integrate Google Maps services** to provide pharmacy locations, open/closed status, routes, ETA, and travel time within the approved scope.
- **Implement a digital ordering process** allowing clients to request medicines from a selected pharmacy.
- **Facilitate prescription management** by allowing clients to upload prescriptions and Pharmacy Staff to verify them when required.
- **Provide pickup and delivery options** according to the client's preference.
- **Support pharmacy-based delivery management** by allowing delivery agents belonging to pharmacies to receive and update delivery assignments.
- **Provide payment management** supporting Mobile Money and Cash on Delivery.
- **Provide order tracking and history** so that clients can monitor their current and previous orders.
- **Provide health information and an advisory symptom checker** while ensuring that the AI feature does not provide medical diagnoses.



III. EXPRESSION OF NEEDS

1. Functional Needs

In this context, functional requirements describe what the **PharmaGo** system should do. The principal modules are organized according to the different actors and system responsibilities.

A. Client

The Client should be able to: **Request an order, make payment, Use the ai symptom checker and geolocation of a pharmacy.**

B. Pharmacy Admin

The Pharmacy Admin should be able to: **Register and manage the pharmacy, Manage stock, Manage pharmacy schedules, Manage delivery agents, Monitor orders.**

C. Pharmacist / Cashier

The Pharmacist/Cashier should be able to: **View incoming order requests, Confirm actual medicine availability, Accept or reject an order, Verify prescriptions, Update order status.**

D. Delivery Agent

The Delivery Agent should be able to: **View assigned deliveries, View delivery information, Update delivery status.**

E. Platform Admin

The Platform Admin should be able to: **Manage user accounts, Manage pharmacies, Monitor orders, Monitor transactions, Manage disputes, Manage health tips, Monitor platform activity.**

External Services



PharmaGo also interacts with external services.

❖ Google Maps API

The system uses Google Maps for the approved geolocation functions: locating nearby pharmacies, obtaining pharmacy location information, providing routes, calculating ETA, providing travel time.

❖ Mobile Money / Payment API Gateway

The payment service processes Mobile Money transactions. The project may use a **sandbox or simulated payment process** for academic demonstration.

2. Non-Functional Needs

Non-functional requirements define the quality characteristics and constraints that PharmaGo must satisfy to ensure that the platform is secure, reliable, usable, and maintainable.

a. Performance Requirements

- The system should respond efficiently to requests from clients, Pharmacy Staff, Delivery Agents, and the Platform Admin.

b. Security Requirements

- User credentials must be securely protected and authentication and authorization mechanisms must be implemented for all user roles, **Role-Based Access Control (RBAC)**.

c. Reliability and Availability

- PharmaGo should remain available to clients and pharmacy personnel under normal operating conditions.
- The system should provide appropriate error handling when network connections, external

3. Chronogram of activities

Table 4: Chronogram of activities

PHASE	OBJECTIVE	OUTPUT	DURATION	PERIOD
INSERTION	Collection of information on the enterprise	Insertion book	2 weeks	03 rd July to 11 th July
EXISTING SYSTEM	Study of the existing system	Existing System	1 week	14 nd July to 22 nd July
SPECIFICATION BOOK	Specification of the user needs	Specification Book	1 week	23 rd July to 30 rd July
ANALYSIS	Capture of needs Use case and textual description Modelling	Analysis Book	2 weeks	31 st July to 13 th August
CONCEPTION	Preliminary conception and Detailed conception	Conception book	1 weeks	14 th August to 26 st August
REALIZATION	Implementation Unitary test Integration Test Development	Realization book	2 weeks	27 th August to 10 th September
TEST OF FUNCTIONALITIES	Testing of the software and debugging	Test of functionalities	2 week	11 th September to 24 th September
INSTALLATION AND USER GUIDE	Documenting software	User Guide	1 week	26 th September to 30 th September

4. Gantt Project Planning

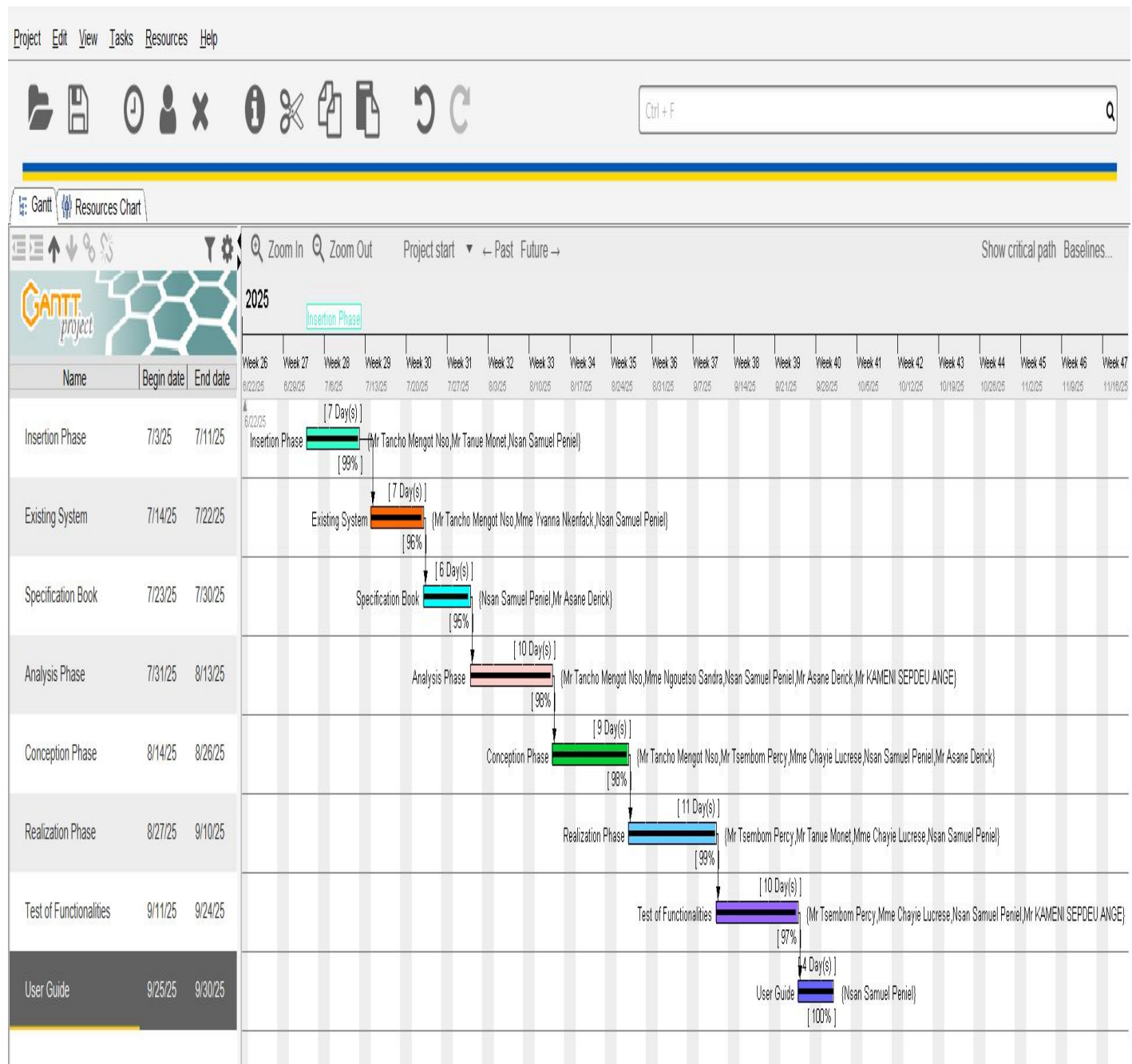


Figure 2: Project Gantt chart

Written By: NSAN SAMUEL PENIEL



IV. PROJECT ESTIMATION COST

A. Software Resources

Table 5 : Software resources (source: MERCURIALE-2025)

RESOURCES	DESIGNATION	USAGE	QUANTITY	UNIT COST (FCFA)
Formatting	MICROSOFT OFFICE PROFESSIONAL 2016	Writing of the report	1	355,465
Web browser	Google Chrome	Access and view web pages on the internet	1	Freeware
Code Editor	Visual Studio Code	For writing the code of the application	1	Freemium
Project Planning	Gantt Project	For building a Gantt chart	1	Freemium
UML Analysis	Visual Paradigm	For drawing UML diagrams	1	Community Version
Version Control	Git & GitHub	For tracking and managing changes to software code	1	Free Version
TOTAL 1			6	355,465

B. Hardware Resources

Table 6 : Hardware resources (source: MERCURIALE-2025)

RESOURCES	HARDWARE	QUANTITY	UNIT COST (FCFA)
Computer	HP ZBook 15 G4 Laptop	1	1,295,000
Printer	IMPRIMANTE HP LASERJET	1	514,528
Network	Local network installation	1	300,000
Smartphone	Techno Spark 20	2	150,000
Total 2		7	2,273,688



DESIGN AND IMPLEMENTATION OF AN AI-ASSISTED PHARMACY STOCK MANAGEMENT PLATFORM WITH GEOLOCATION-BASED DRUG SEARCH AND DELIVERY SERVICE



C. Human Resources

Table 7 : Human resources (source: Mercurial 2025)

ROLE	NUMBER OF DAYS	QUANTITY	COST PER DAY	TOTAL PRICE (FCFA)
Project Manager	90	01	30,000	2,700,000
Analyst	21	01	25,000	525,000
UI/UX Designer	30	01	10,000	300,000
Programmer	30	01	15,000	450,000
Tester	14	02	10,000	280,000
Margin error	/	/	/	1,475, 000
TOTAL 3				5,730,000

D. Global Estimation

Table 8 : Global estimation

TOTAL 1 (FCFA)	TOTAL 2 (FCFA)	TOTAL 3 (FCFA)	OVERALL TOTAL (FCFA)
355,465	2,273,688	5,730,000	8,359,153
EIGHT MILLION THREE HUNDRED FIFTY-NINE THOUSAND ONE HUNDRED FIFTY-THREE			

Written By: NSAN SAMUEL PENIEL



V. PROJECT CONSTRAINTS

1. Technical Constraints

The project's technical feasibility is constrained by our dependency on the municipal service's infrastructure; the core-tracking feature requires their vehicles to be equipped with GPS and their

2. Time Constraints

The project must be delivered by October 2nd, 2026, following a 13-week (3-month) development cycle starting July 3rd, 2026.

3. Cost constraint

The implementation of our project will necessitate expenses on work force, materials, and software, amounting to 8,359,153.

VI. LIST OF PARTICIPANTS AND DELIVERABLES

1. LIST OF PARTICIPANTS

Table 9 : List of participants

NAME	FUNCTION	ROLE
Mr. MENGOT NSO	Follows up interns at the company level	Professional Supervisor
Mr. ASANE DERICK	Follows up student at the academic level	Academic Supervisor
NSAN SAMUEL PENIEL	AICS Student Intern	Student at AICS

Written By: NSAN SAMUEL PENIEL



2. DELIVERABLES

In project management, any component materializing the result of a realization service is a deliverable. In the case of our project, the deliverables are:

A report is composed of the following document:

- ❖ A User Guide
- ❖ Software Application
- ❖ PowerPoint



CONCLUSION

In conclusion, this part has come to a close, wherein we have effectively listed and expounded on our goals. The specifications book provided us with the means to present the project's different stakeholders, as well as the prerequisites and tentative schedule essential for its realization. Moving forward, we will directly proceed with the analysis phase, where we will utilize a modelling language and a unified approach to exemplify our system. Furthermore, we will carry out a comparative evaluation of UML and Merise methodologies.



FILE 3: ANALYSIS PHASE



Preamble

The main objective of the analysis phase is to capture the user's need, the delimitation of the field of study and to have a clear understanding of the system in study. To achieve this, we will use UML (Unified Modelling Language) with the 2TUP (two Track Unified Process) as method applied to UML to analyze the system. We will start by performing a preliminary study of the software, followed by the identification of the different stakeholders that interact with the system, finally perform the analysis of the functional needs through a global and detail view of the application.

Content Overview

INTRODUCTION

I. COMPARATIVE METHODOLOGY

1. STUDY OF UML AND MERISE
2. STUDY OF UNIFIED PROCESSES

II. CHOICE OF THE ANALYSIS METHOD

III. JUSTIFICATION OF THE ANALYSIS METHOD

IV. MODELLING OF THE PROPOSED SOLUTION

CONCLUSION



INTRODUCTION

System development can be thought of as having two major components: System analysis and system design which both help in understanding the details of the existing system or the system to be designed. The analysis and design of information systems has most of the time vocation to allow the creation of databases, which must represent as closely as possible the reality of the field studied thus requiring the use of a design method. This is why our choice will be directed on the UML method as it offers much to developers seeking a user-centered approach and / or a wide scope in design. This part of the report consists of the comparative study of UML and MERISE unified processes and finally the various diagrams that meet the functional need requirements.



I. COMPARATIVE METHODOLOGY

A. STUDY OF UML AND MERISE

i. MERISE

MERISE stands for “Méthode d’Etude et de Réalisation Informatique pour des Systèmes d’Entreprise”. Although it is prescriptive to some extent, MERISE permits the end users and senior management. MERISE is a method for designing, developing and carrying out IT projects.

ii. UML

UML (Unified Modelling Language) is a standard notation for the modelling of real-world objects as a first step in developing an object-oriented design methodology captures information about the static and dynamic view of a system. UML 2.0 comprises of 13 diagrams which represent the different views of a system.

1. STATIC OR STRUCTURAL DIAGRAMS

- ❖ Class diagram
- ❖ Object diagram
- ❖ Component diagram
- ❖ Deployment diagram
- ❖ Package diagram
- ❖ Profile Diagram

2. BEHAVIOURAL OR DYNAMIC DIAGRAMS

- ❖ Use case diagrams.
- ❖ Activity diagram.
- ❖ State machine diagram.
- ❖ Sequence diagram.
- ❖ Communication diagram
- ❖ Global Interaction diagram.
- ❖ Timing Diagram

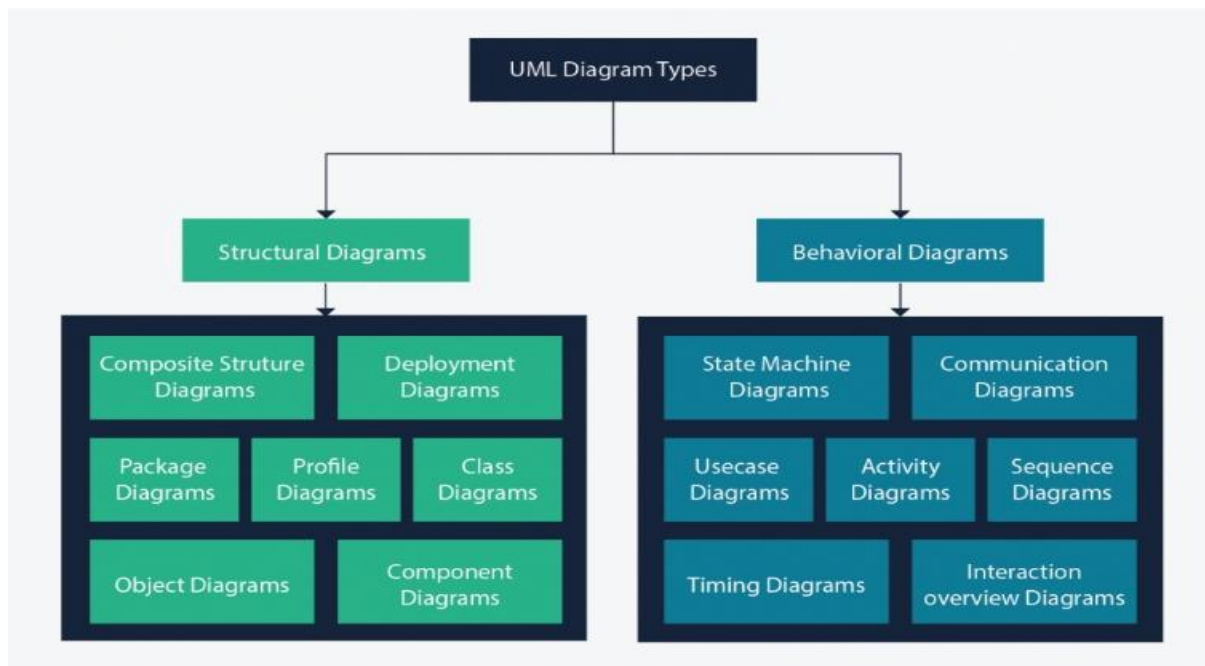


Figure 3: UML 2.5 diagrams overview (source: <https://creately.com/blog/diagrams/uml-diagram-types-examples/>)

1. STUDY OF UNIFIED PROCESSES

A. Unified Process (UP)

A Unified Process is a generic software development method. Generic means that it is necessary to adapt UP to the context of the project, team, domain and/or organization.

- ✚ **Iteration** is distinct sequence of activities with a basic plan and evaluation criterion that produces an internal or external output.
- ✚ **An increment** is the difference between two released products at the end of two iterations. Each iteration that the group is capable of integrating the technical environment.
- ✚ **Centered on architecture** the different models derived during the establishment of system must be reliable and coherent.
- ✚ **Driven by use case and requirements** enables the clear definition of a user' needs and priorities respectively thereby minimizing the risk of project failure.



B. Two Track Unified Process (2TUP)

2TUP is a unified process, which is built on UML and has as objective to bring solution to constraints of functional and technical changes imposed on information systems by strengthening controls on development capacities.

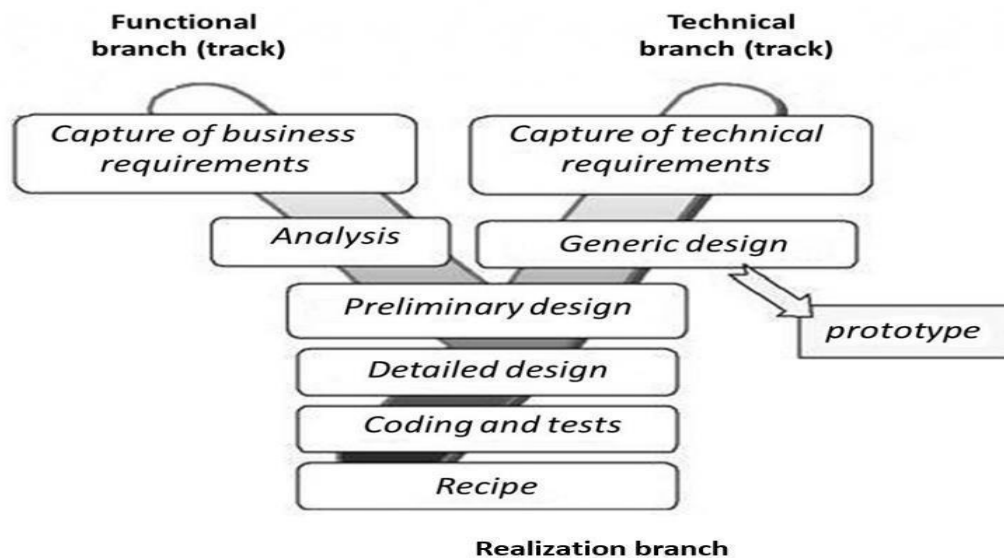


Figure 4: 2TUP diagram (Source: <https://www.mysciencework.com/omniscience/pervasive-mobile-healthcare-system-basedon>)

a. The left branch (functional branch)

It captures the functional needs of a system.

b. The right branch (Technical branch)

The technical branch enumerates the technical needs and proposes a generic design validated by a prototype.

c. The middle branch (Realization or Implementation branch)

In this branch, we study the preliminary conception, detailed conception, and documentation of the system. The realization branch supports the following:

- **Preliminary conception:** This is the most sensitive step of 2TUP as it is the confluence of the functional and technical branch. It is completed when the deployment model, the operating model, the logical model, interphases and the software configuration model are defined. We have the following diagrams:



DESIGN AND IMPLEMENTATION OF AN AI-ASSISTED PHARMACY STOCK MANAGEMENT PLATFORM WITH GEOLOCATION-BASED DRUG SEARCH AND DELIVERY SERVICE



- **Detailed conception:** This is the detailed design of each feature of the system.
- **Coding and testing:** This is the phase where we program the designed and coded features.
- **The recipe:** Also known as, the deliverables is the validation phase of the functions of the developed system.

CHARACTERISTICS OF 2TUP

The two-track unified process was built to be used with the Unified Modeling Language to become a method and has the following characteristics:

- **It is user oriented:** 2TUP is built from user's expectation.
- **It is component oriented:** it offers flexibility to the model by supporting the re-use of components.
- **It is an iterative process:** processes are being done in iterations and each iteration shows a precise level of abstraction.
- **It is an incremental process:** allowing a better functional and technical risk management and thus constituting the deadlines and the cost control.



II. CHOICE OF THE ANALYSIS METHOD

On very important notice is that UML is not a method but a modelling language. There exist several Unified Processes but our modelling approach will be the 2TUP (Two-track unified process) which we will use in the course of our project.

Table 10 : Comparison between MERISE and UML

MERISE	UML
It is based on procedural approach	Based on object approach
Suitable for relational databases design and implementation	Suitable for object-oriented design and implementation
It stands for Méthode d'Étude et de Réalisation Informatique pour les Systèmes d'Entreprises	Unified Modeling Language
MERISE is a systemic method of analysis and design of information systems. That is, it uses a systems approach.	UML is however not a method but rather an object modeling language to which it is necessary to associate an approach to make it a method. This is the case with the 2TUP method: RUP and XP.
Merise is a method of analysis	UML is modeling language
Emphasizes data analysis and modelling using data flow diagrams (DFD), entity-relationship diagrams (ERD), and data dictionaries.	Emphasizes system behaviour and structure using 14 diagram types, such as use case diagrams, class diagrams, and activity diagrams



III. JUSTIFICATION OF THE ANALYSIS METHOD

The reason why we chose UML modelling language and the software development process 2TUP instead of many others that exist, include:

- UML is the current standard for programming in an object-oriented language. For this reason, it is widely understood and well known making it easy for a new programmer to join the project and be productive from the very first day.
- UML diagrams allow teams to virtualize how a project is or will be working, and they can be used in any field, not just software engineering. The diagrams will allow teams to virtualize together how a system or a process will work or did work. It can provide new ideas for how teams have to collaborate to achieve the goal of the workflow process.
- 2TUP is centered around the creation and maintenance of a model, rather than the production of mountain documents.
- 2TUP is user oriented as it permits the development of software that responds to the needs of the users through the study of the user needs.
- 2TUP is iterative and incremental, hence it enables the project team to produce refined amelioration if necessary and easily integrate it in the already existing system.
- 2TUP by permitting the project team identify and test the key functionalities of the system limits the risk related to building the system.

IV. MODELING OF THE PROPOSED SOLUTION

1. USE CASE DIAGRAM

a. Definition

Use case diagram shows the functionalities of a system, their interdependencies and how they relate with actors of the system. A use case is a specification of behavior. The main objectives of the use case diagram are:

- Provide a high-level view of the system.
- Identify the functions of the system.

Use case diagrams are completed with a textual description of each use case that is intended to define the use case in details.

b. Formalism

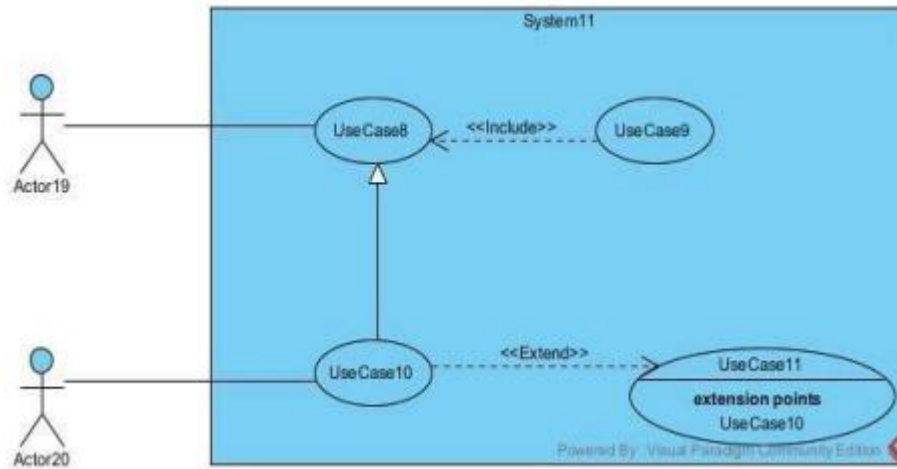
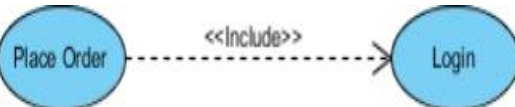
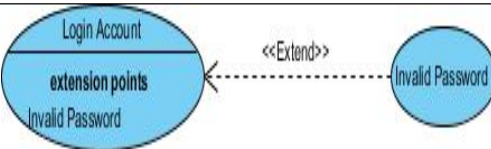


Figure 5: Use case formalism



c. Components of use case diagram

Table 11 : Component of use case diagram

Element	Notation	Description
Actors	 Actor	Represents an entity that directly interacts with the system. The actor is what performs the different possible actions of the system
Use case	 UseCase	A use case represents a functionality of the system. An actor can perform an action.
Association	 Student	It indicates that an actor takes part in a use Case.
Include	 Place Order	An inclusion denotes that an included action must be performed before the including action can be performed.
Extend	 Login Account	An extension denotes that an extending action may be performed while an extended action is being performed.
Generalization	 Store Patient Records(Paper File)	This shows that an actor or a use case is a kind of other abstract or concrete actors, which can be defined and later specialized using generalization relationship.
System	 System	It is a container of use cases which interact with external actors



DESIGN AND IMPLEMENTATION OF AN AI-ASSISTED PHARMACY STOCK MANAGEMENT PLATFORM WITH GEOLOCATION-BASED DRUG SEARCH AND DELIVERY SERVICE



d. Actors of our system

Table 12 : Actors of our system

Actor	Role
Client	Login/Register, activate GPS, search for a drug, view nearby pharmacies and availability, select pharmacy, place order, upload prescription, choose pickup/delivery, make payment, track order, receive medicine, use AI symptom checker, view health tips
Delivery Agent	Receives assignment, picks medicine, delivers it, updates delivery status
Pharmacist / Cashier	Receives order, checks actual stock, confirms/rejects order, verifies prescription
Pharmacy Admin	Registers and manages pharmacy information, manages pharmacy stock, adds/updates/removes medicines, manages pharmacy delivery agents, manages delivery schedule/window
Super Admin	Manages user accounts, manages pharmacies, monitors orders and payments, manages disputes, views platform analytics, manages health tips
Gemmi AI	Provides an advisory symptom checker , analyzes user-provided symptoms and provides general health guidance; does not provide a medical diagnosis
Geolocation API	Location, nearby pharmacy information, route, ETA/travel time
Payment API	Processes Mobile Money payment



e. General Use Case Diagram

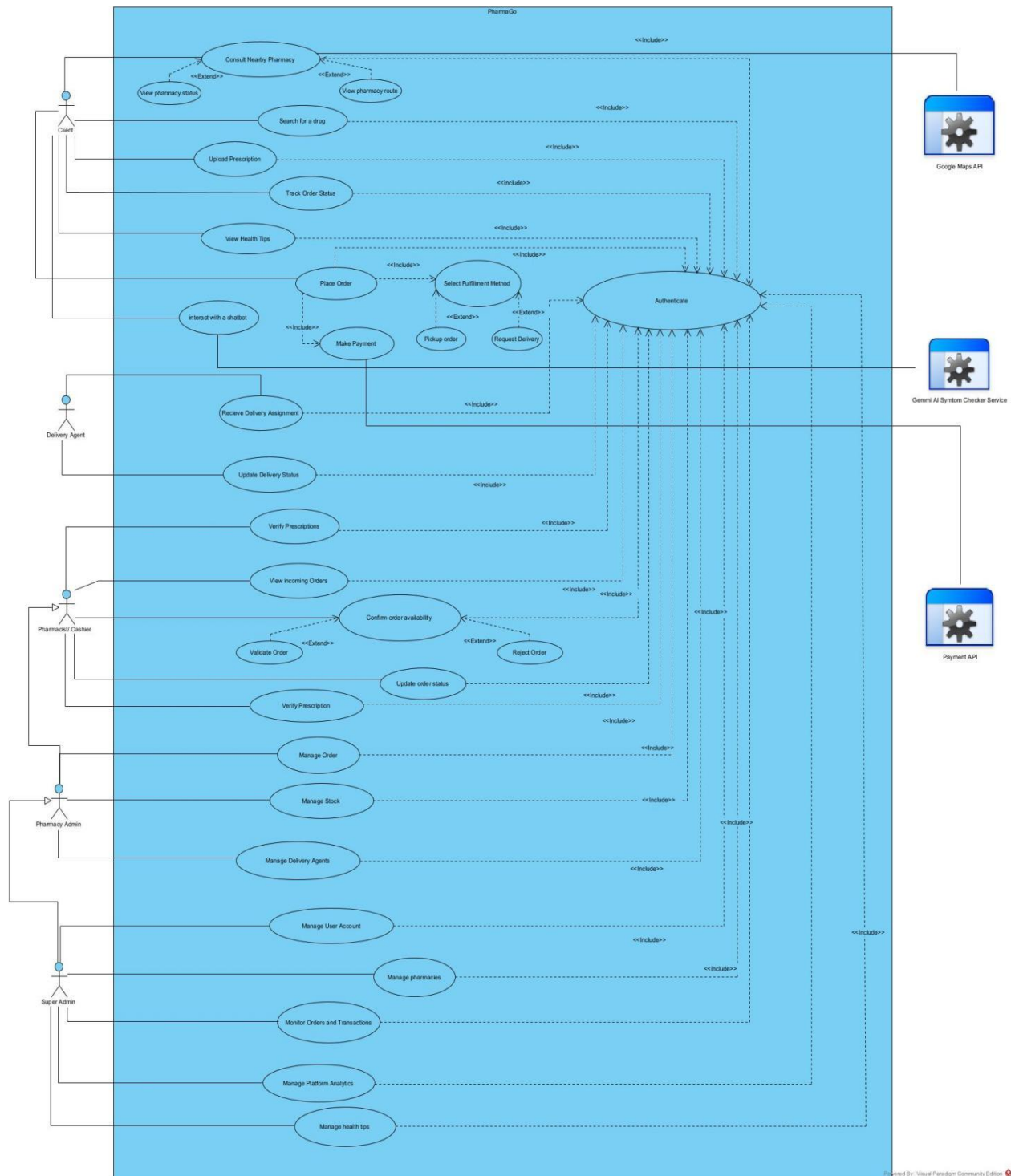


Figure 6: General use case diagram Zerodech



i. Textual Description

UML allows the execution of a use case to be described in a textual way, in a form called nominal scenario. A nominal scenario describes in more detail the execution of a use case by an actor until it is successfully completed.

ii. Formalism

A textual description of a use case is represented in the following form

Table 13 : Textual description formalism

NAME OF THE USE CASE	
Summary	Aim of the use case
Actors	Users
Pre-condition	Set of actions that must be completed before the launching of the use case
Trigger	The event that causes the use case to be initiated
Nominal scenario	It corresponds to a normal development of a use case
Alternative scenario	These are variants of the nominal scenario.
Post condition	Set of mechanism that can lead to the end of the use case

1. Create Account

Written By: NSAN SAMUEL PENIEL



DESIGN AND IMPLEMENTATION OF AN AI-ASSISTED PHARMACY
STOCK MANAGEMENT PLATFORM WITH GEOLOCATION-BASED
DRUG SEARCH AND DELIVERY SERVICE



Table 14 : Textual description of Create account

Field	Description
Name of the Use Case	Create Account
Summary	Allows a new client to register in the system to gain access to waste management services.
Actors	Client (new user)
Pre-condition	User has internet access and a device.
Trigger	User selects <i>Create Account</i> on the app/website.
Nominal Scenario	1. User clicks <i>Create Account</i>. 2. System displays registration form. 3. User enters name, email/phone, and password. 4. System validates input. 5. Account created and saved. 6. Confirmation displayed.
Alternative Scenario	7. Input invalid (weak password, duplicate email/phone) → error displayed. 8. Validates and stores new user record; sends confirmation.
System Response	Client account Created
Post-condition of success	Client has a valid account and can log in.
Post-condition of failure	Client not log in and error message

Written By: NSAN SAMUEL PENIEL



2. Search drug and pharmacy

Table 15 : Textual description of search drug and pharmacy

Field	Description
Name of the Use Case	Search Drug and Pharmacy
Summary	Allows a client to search for a medicine and identify nearby pharmacies where the medicine is listed as available.
Actors	Client, Geolocation API
Pre-condition	Client has a valid PharmaGo account, is logged in, has an internet connection, and has activated GPS/location services.
Trigger	Client enters the name of a medicine in the search bar and submits the search.
Nominal Scenario	1. Client opens the drug search feature. 2. System requests the client's location. 3. Client activates GPS and grants location permission. 4. Client enters the name of the desired medicine. 5. System validates the search request. 6. System searches the database for pharmacies listing the requested medicine in their stock. 7. System identifies nearby pharmacies using the client's location. 8. System displays the matching pharmacies and the medicine's listed availability. 9. Client selects a pharmacy to view its details. 10. Client can proceed to Place Order.
Alternative Scenario	1. GPS is disabled → System asks the client to activate GPS. 2. Location permission is denied → System informs the client that location access is required. 3. Medicine is not found → System displays Medicine not found. 4. No nearby pharmacy lists the medicine → System displays No nearby pharmacy found with this medicine. 5. Search information is invalid → System asks the client to enter a valid medicine



	name. 6. Database or Geolocation API unavailable → System displays an error and asks the client to try again.
System Response	Displays nearby pharmacies where the requested medicine is listed as available, together with relevant pharmacy information and location.
Post-condition of success	Client identifies a suitable pharmacy and can proceed to place an order for the medicine.
Post-condition of failure	No matching pharmacy is found, and the client receives an appropriate notification.

3. Place Order

Table 18 : Textual description of Make Payment

Field	Description
Name of the Use Case	Place Order
Summary	Allows a client to place an order for a selected medicine from a nearby pharmacy through the PharmaGo platform.
Actors	Client, Pharmacist/Cashier
Pre-condition	Client has a valid PharmaGo account, is logged in, and has selected a pharmacy and medicine.
Trigger	Client selects a medicine and clicks Place Order .
Nominal Scenario	1. Client selects a pharmacy and medicine. 2. Client clicks Place Order .

Written By: NSAN SAMUEL PENIEL



	3. System displays the order details. 4. Client confirms the order request. 5. System creates and saves the order. 6. System sends the order to the Pharmacist/Cashier for verification. 7. Pharmacist/Cashier checks the actual stock. 8. Pharmacist/Cashier confirms the order. 9. System updates the order status to Confirmed. 10. Client receives the order confirmation.
Alternative Scenario	1. Required medicine is not actually available → Pharmacist/Cashier rejects the order. 2. Prescription is required but not provided → System requests prescription upload. 3. Order information is incomplete or invalid → System displays an error and asks the client to correct it.
System Response	Order is created, sent for pharmacy verification, and, if accepted, marked as Confirmed .
Post-condition of success	Client has a confirmed order and can proceed to choose pickup or delivery .
Post-condition of failure	Order is not confirmed and the client receives an appropriate notification explaining the reason.



2. Activity Diagram

An activity diagram provides a behavior view of the system by describing the sequence of action in a process or a particular activity. The activity diagram makes it possible to emphasize the processing and to graphically formalize the sequence of action carried out in the use case.

a. Formalism

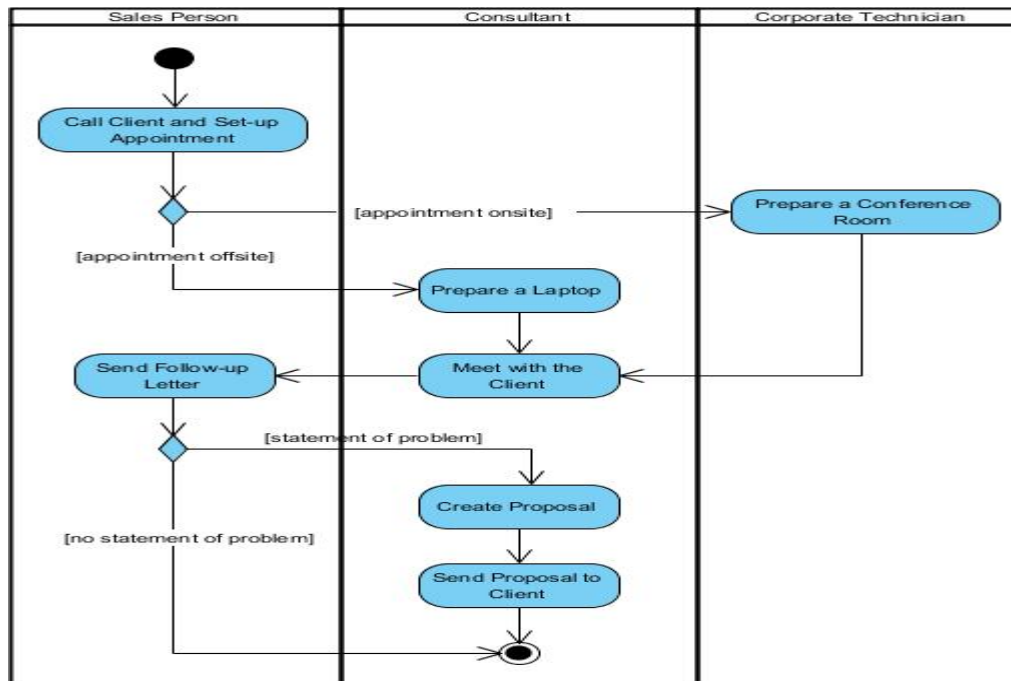
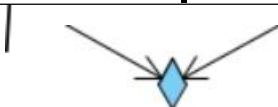
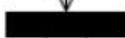
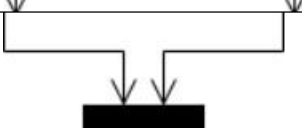


Figure 7: Activity diagram formalism

c. Component of an Activity diagram

Table 19 : Component of an Activity diagram

Element	Notation	Description
Activity		Is used to represent a set of actions
Action		A task to be performed
Activity Edge		An activity edge in a UML activity diagram is a directed connection between two activity nodes.
Initial node		Portrays the beginning of a set of actions or activities
Activity Final Node		Stop all control flows and object flows in an activity (or action)
Object node		Represent an object that is connected to a set of Object Flows
Decision node		Represent a test condition to ensure that the control flow or object flow only goes down one path
Merge node		Bring back together different decision paths that were created using a decision-node.
Fork node		Split behavior into a set of parallel or concurrent flows of activities (or actions)
Join node		Bring back together a set of parallel or concurrent flows of activities (or actions).
Swimlane and Partition		A way to group activities performed by the same actor on an activity diagram or to group activities in a single thread



d. Activity diagram of Authenticate

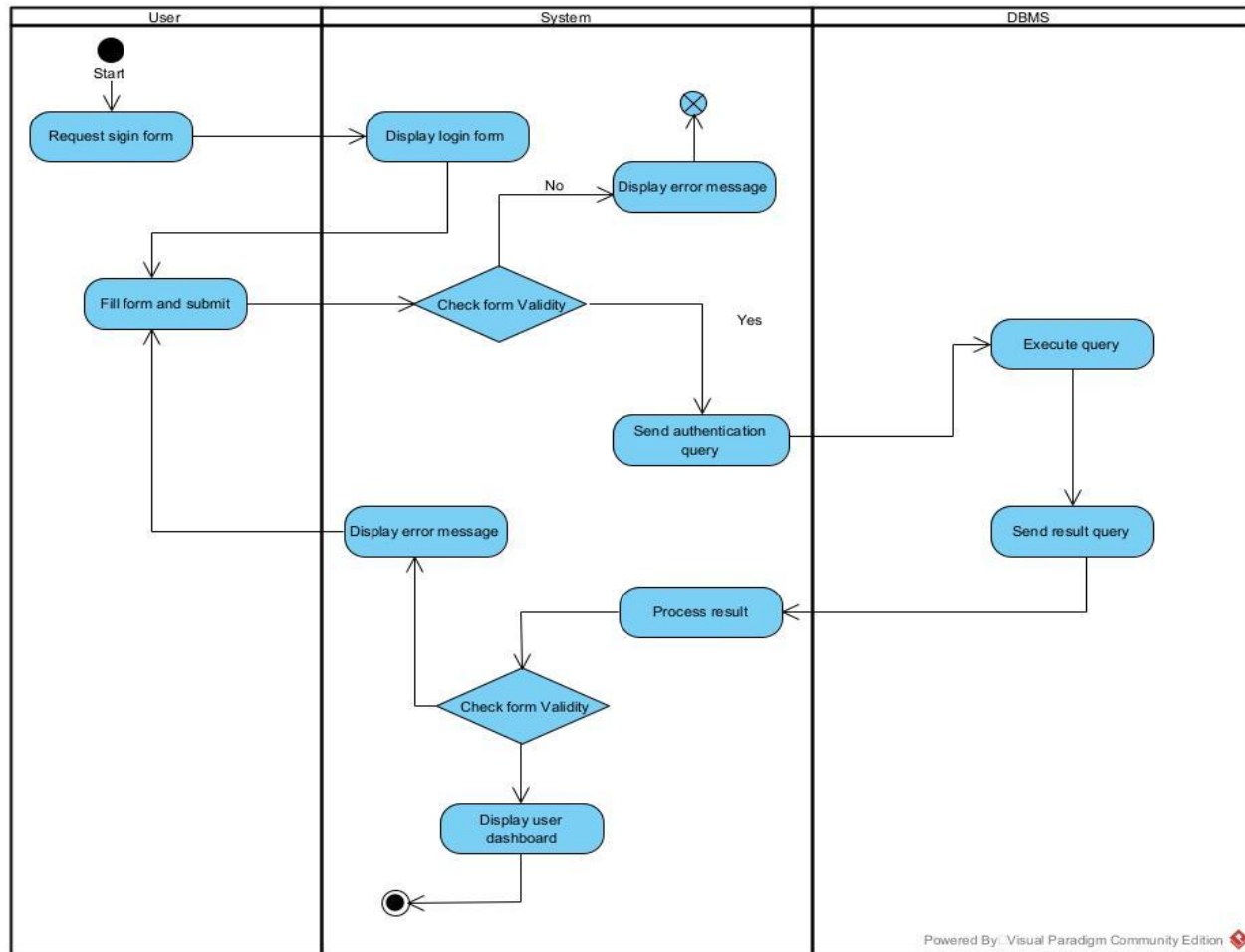


Figure 8: Authenticate activity diagram

e. Activity diagram of Account Creation

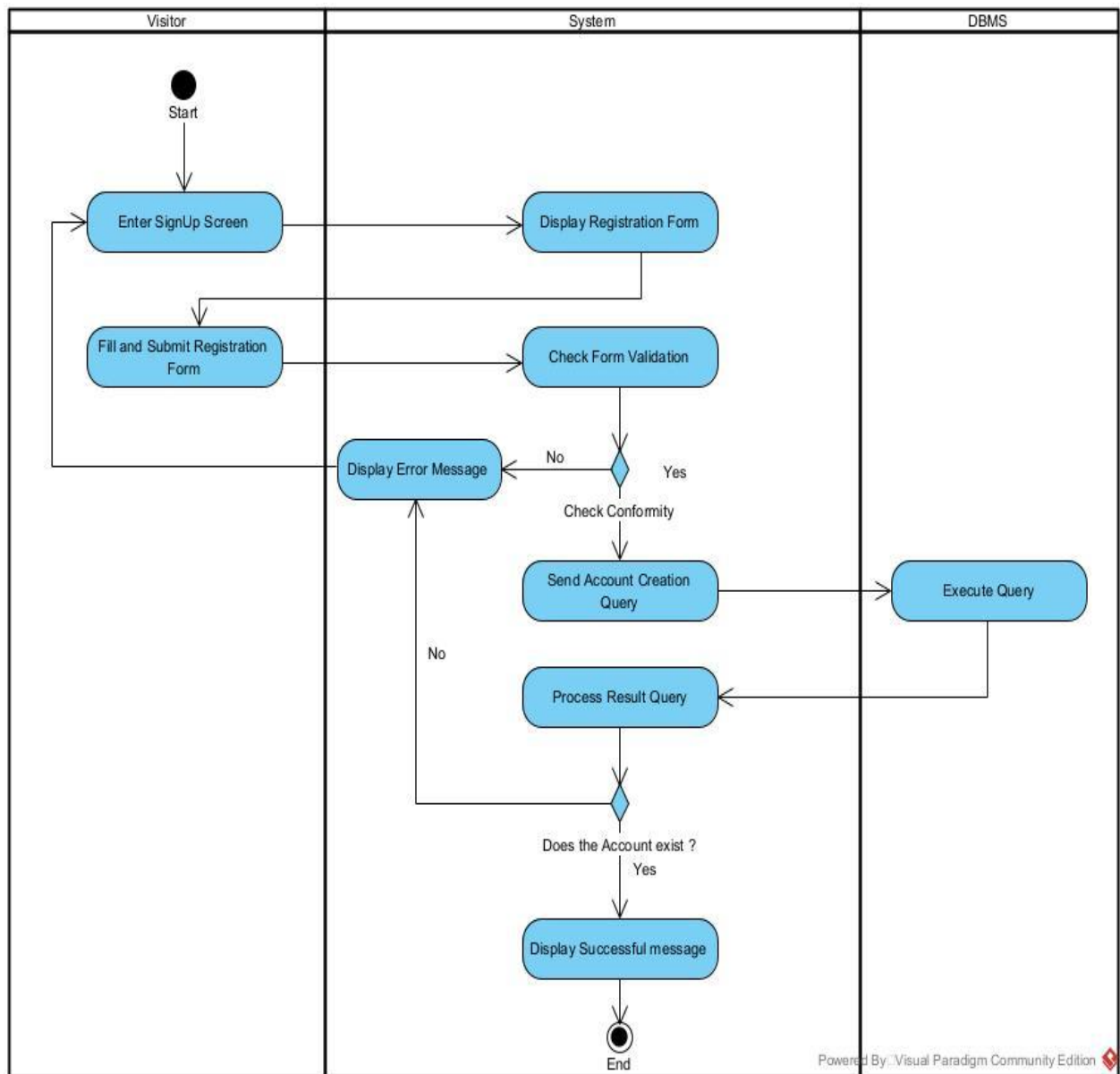
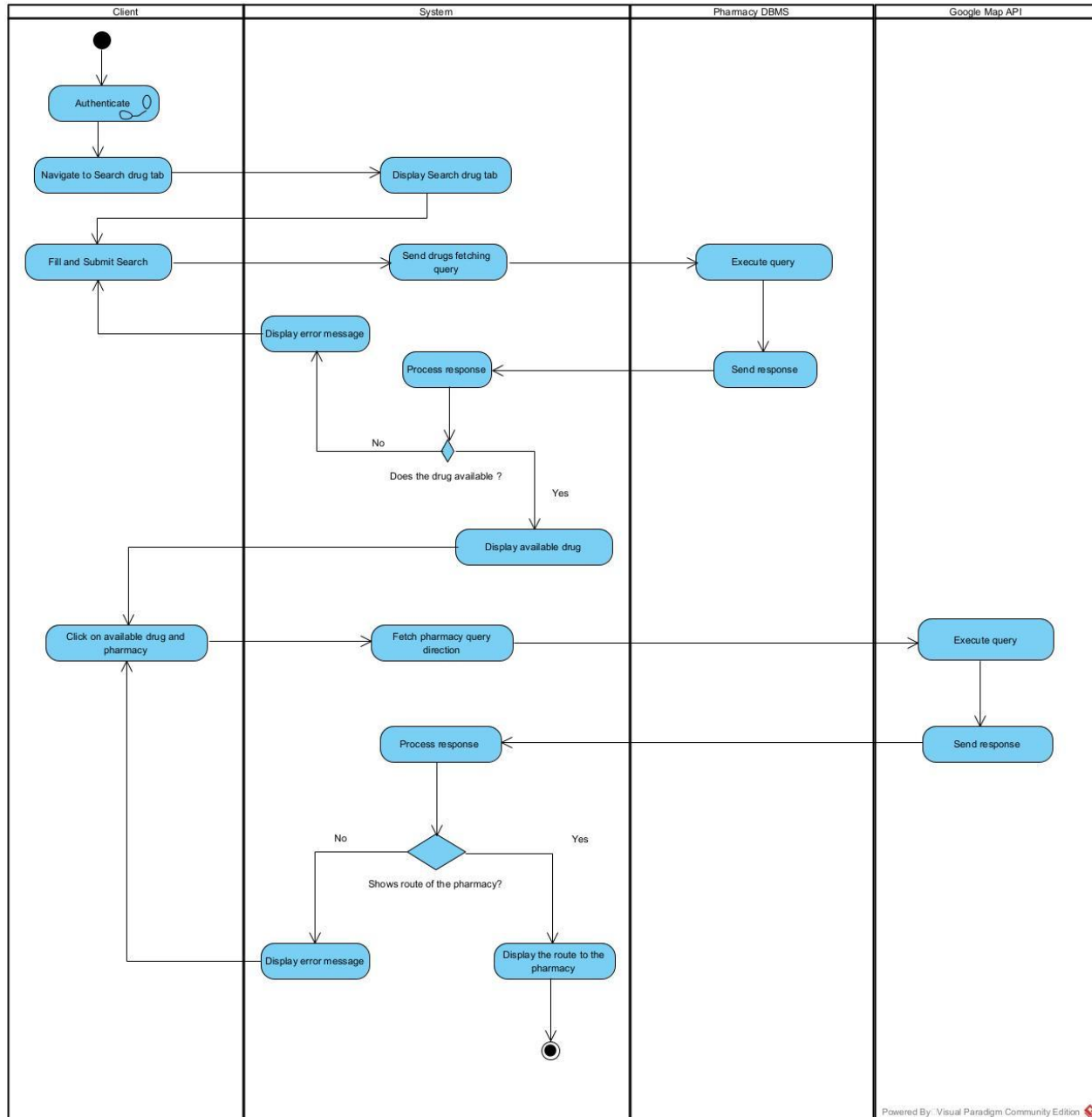


Figure 9: Create account activity diagram



f. Activity diagram of Search Drug and Pharmacy





g. Activity diagram of Place Order

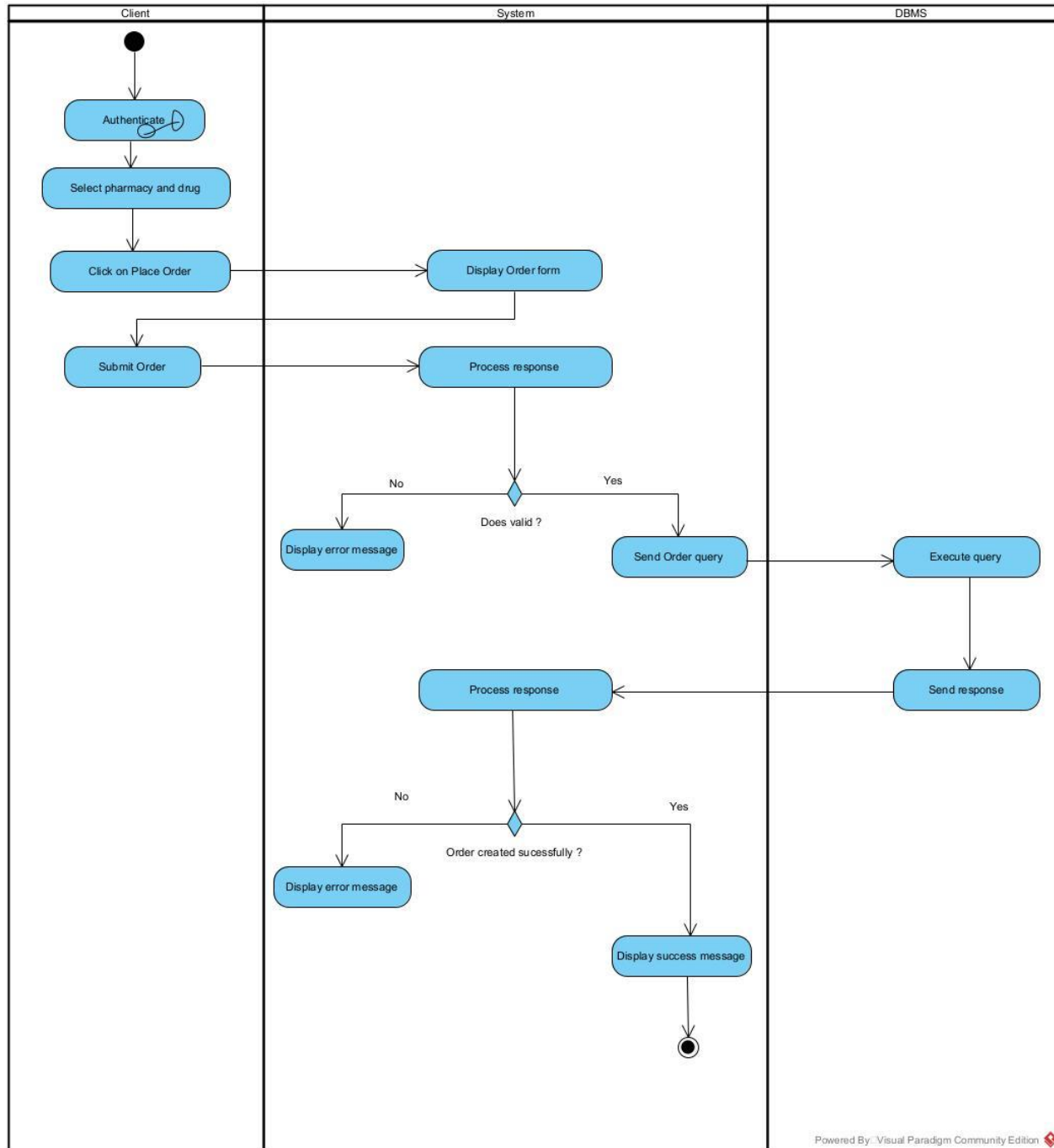


Figure 10: activity diagram

Written By: NSAN SAMUEL PENIEL



3. SEQUENCE DIAGRAM

a. Definition

A Sequence diagram describes interactions among classes in terms of an exchange of messages over time. They are also called event diagrams. A Sequence diagram is a good way to visualize and validate various runtime scenarios. These can help to predict how a system will behave and to discover responsibilities a class may need to have in the process of modelling a new system.

b. Formalism

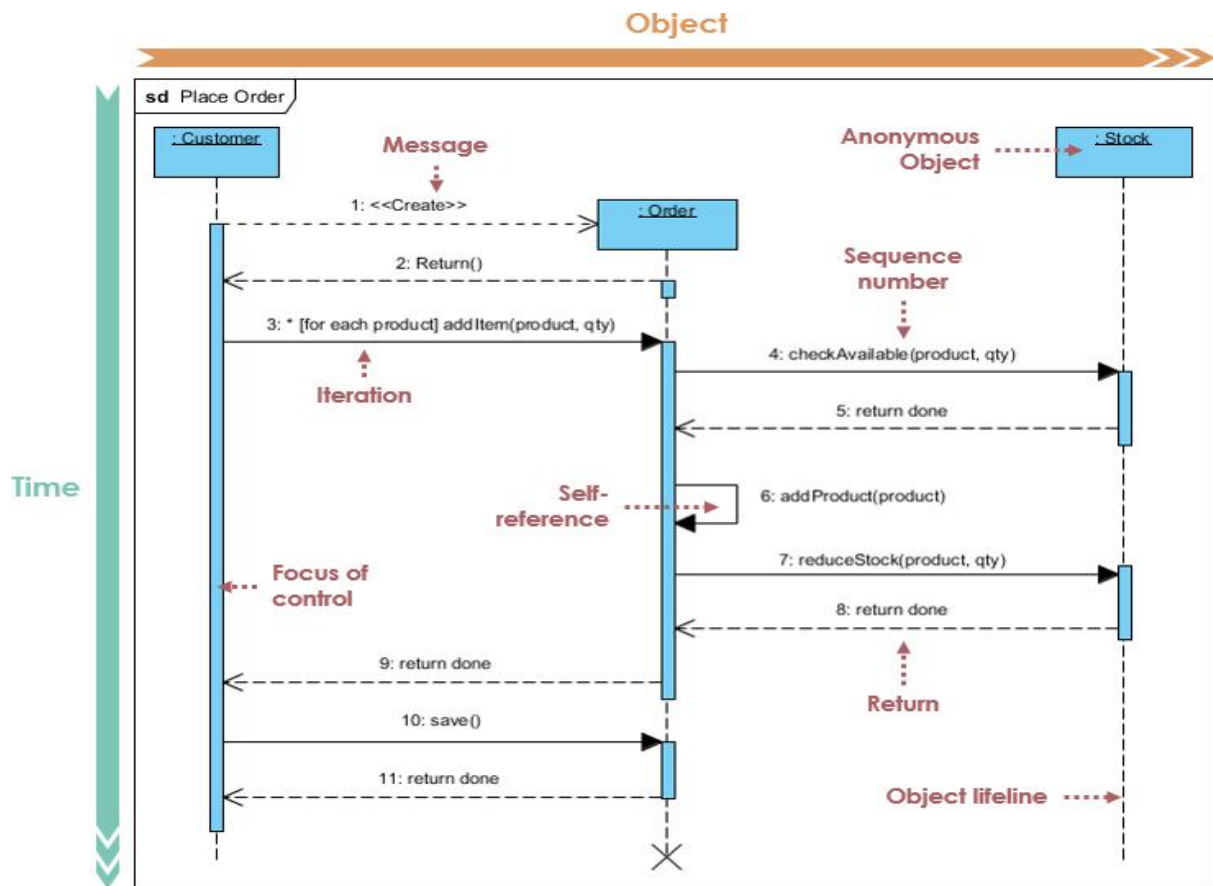
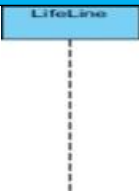
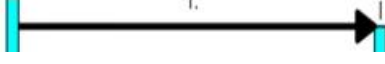
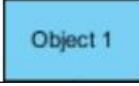


Figure 12: Sequence diagram formalism



c. Component of sequence diagram

Table 20 : Component of sequence diagram

Element	Notation	Description
Lifelines		They represent rows or objects instances that participate in the sequence being modelled.
Asynchronous Message		It is a message that receives an indirect response.
Synchronous Message		It is a message that sends and want response before it continues a process
Self-Message		Self-message is a kind of message that represents the invocation of message of the same lifeline.
Return Message		It represents the response of a message.
Actor		They send and receive message.
Combined Interaction Fragment		An articulation of interaction diagram, defined by an operator and operands.
Object		They send and receive messages



d. Sequence diagram of Authenticate

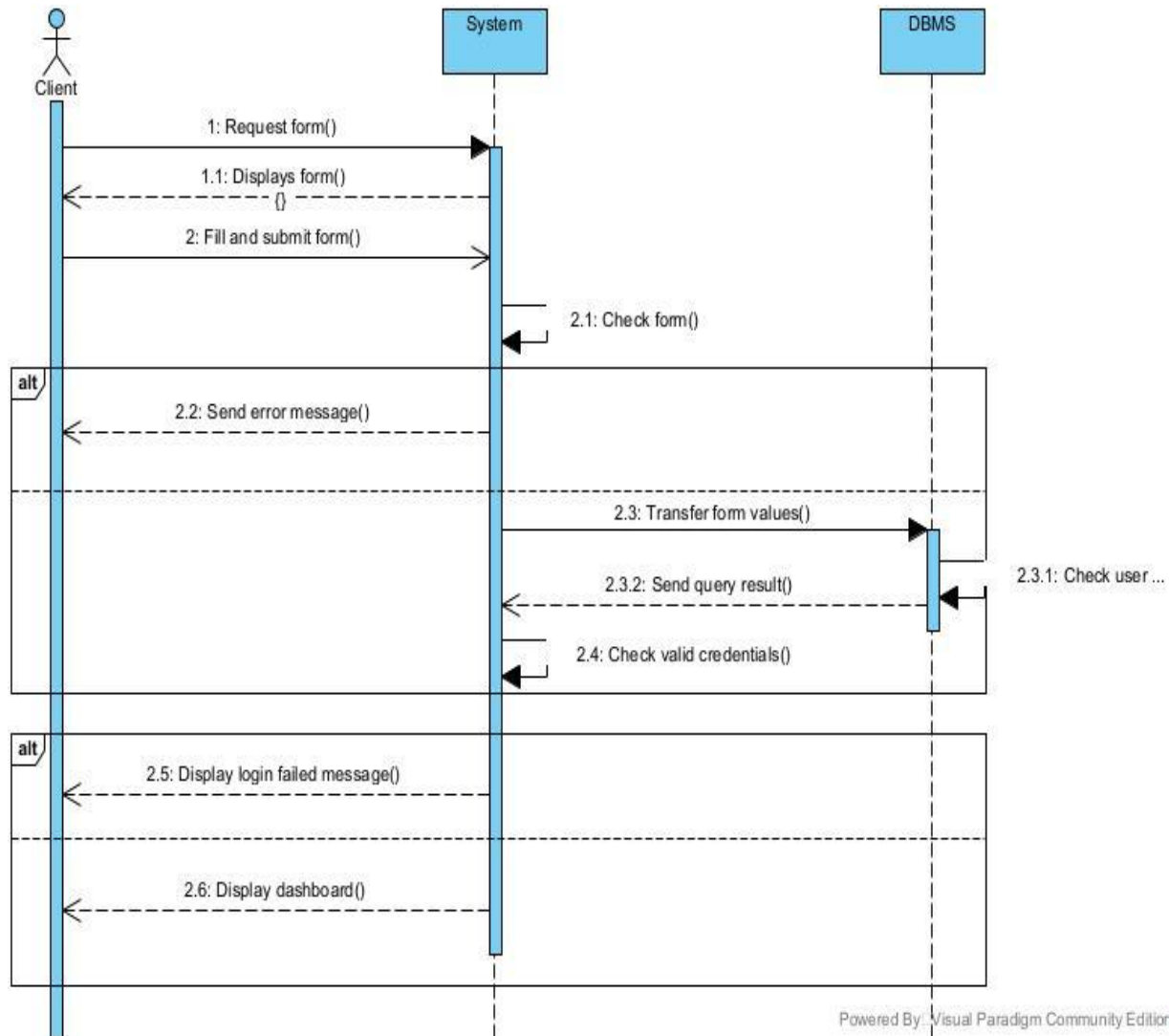


Figure 13: Sequence diagram of Authenticate



e. Sequence diagram of Search Drug and Pharmacy

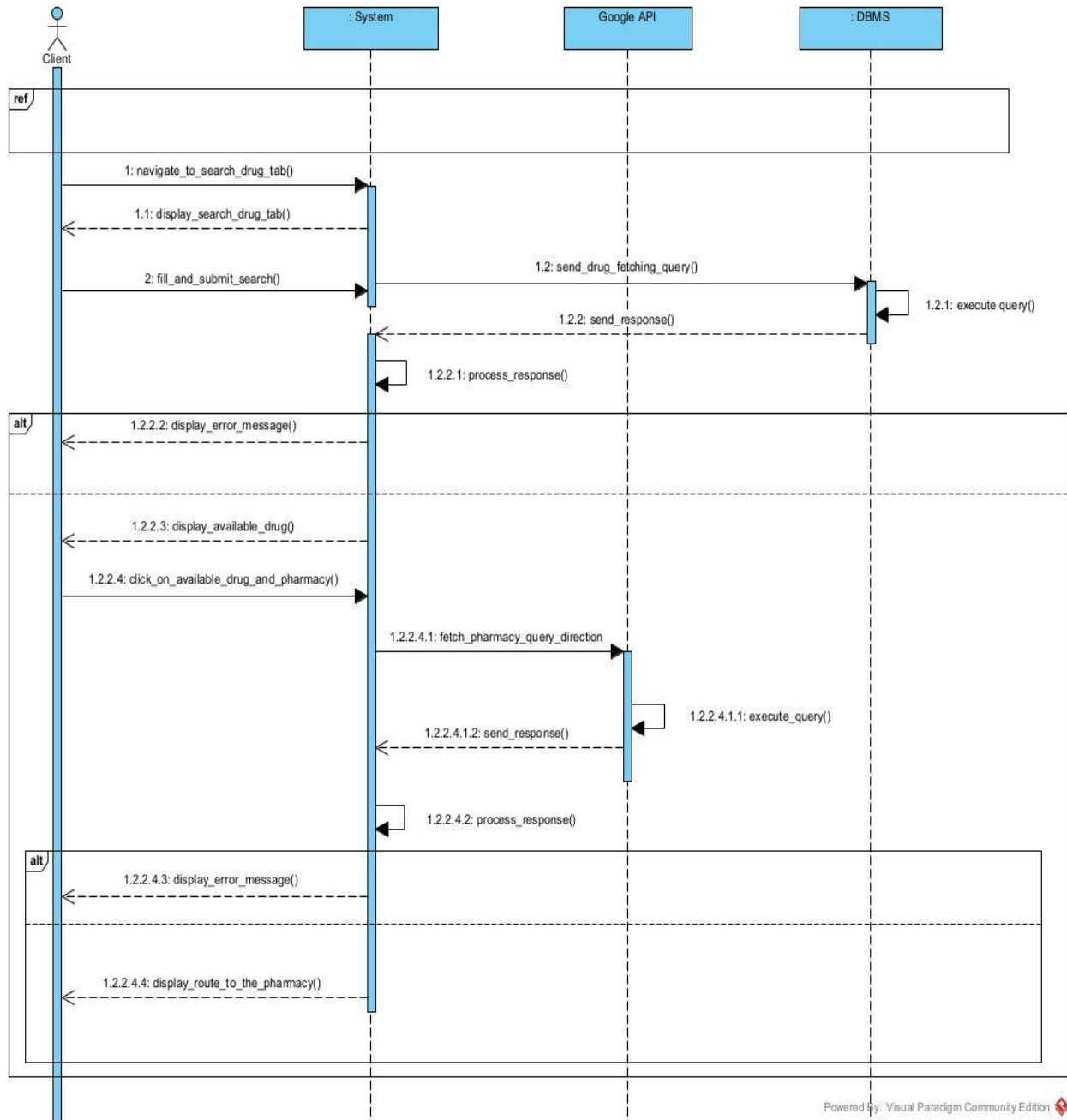


Figure 15: Sequence diagram of



f. Sequence diagram of Place Order

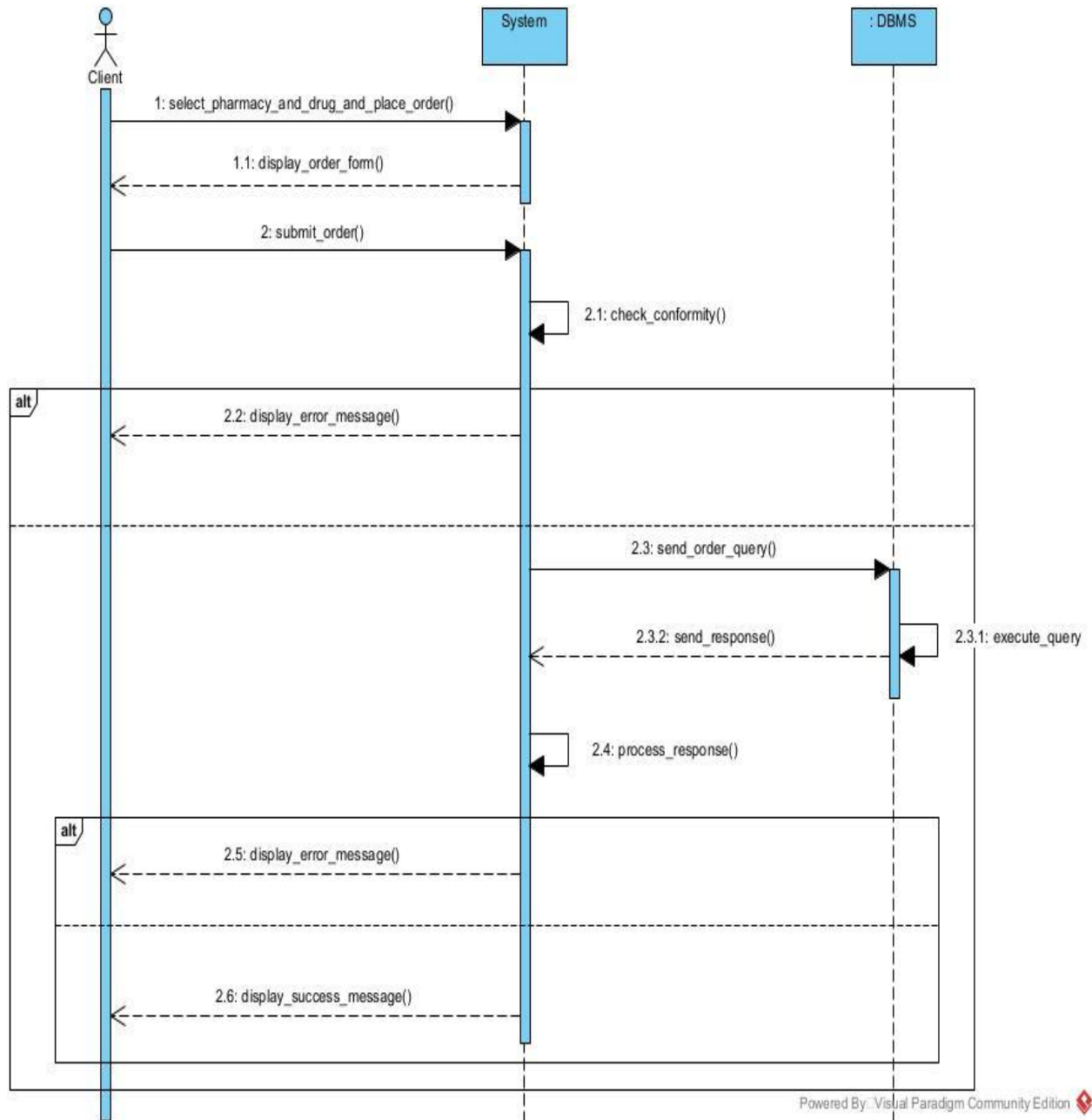


Figure 16: Sequence diagram of

4. COMMUNICATION DIAGRAM

a. Definition

Communication Diagrams model the interactions between objects in a sequence. They describe both the static structure and the dynamic behavior of a system. It is a simplified version of a Collaboration Diagram introduced in UML 2.0. A communication diagram is more focused on showing the collaboration of objects rather than the time sequence.

b. Formalism

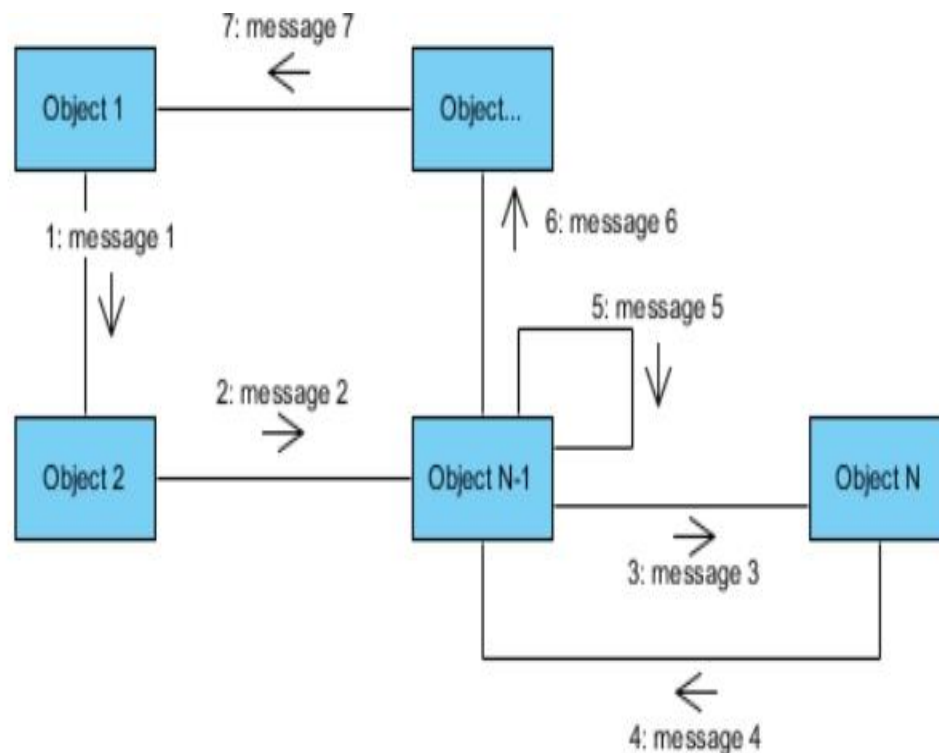
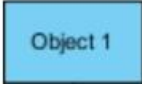


Figure 18: Communication diagram formalism

c. Components of a communication diagram

Table 21 : Component of a communication diagram

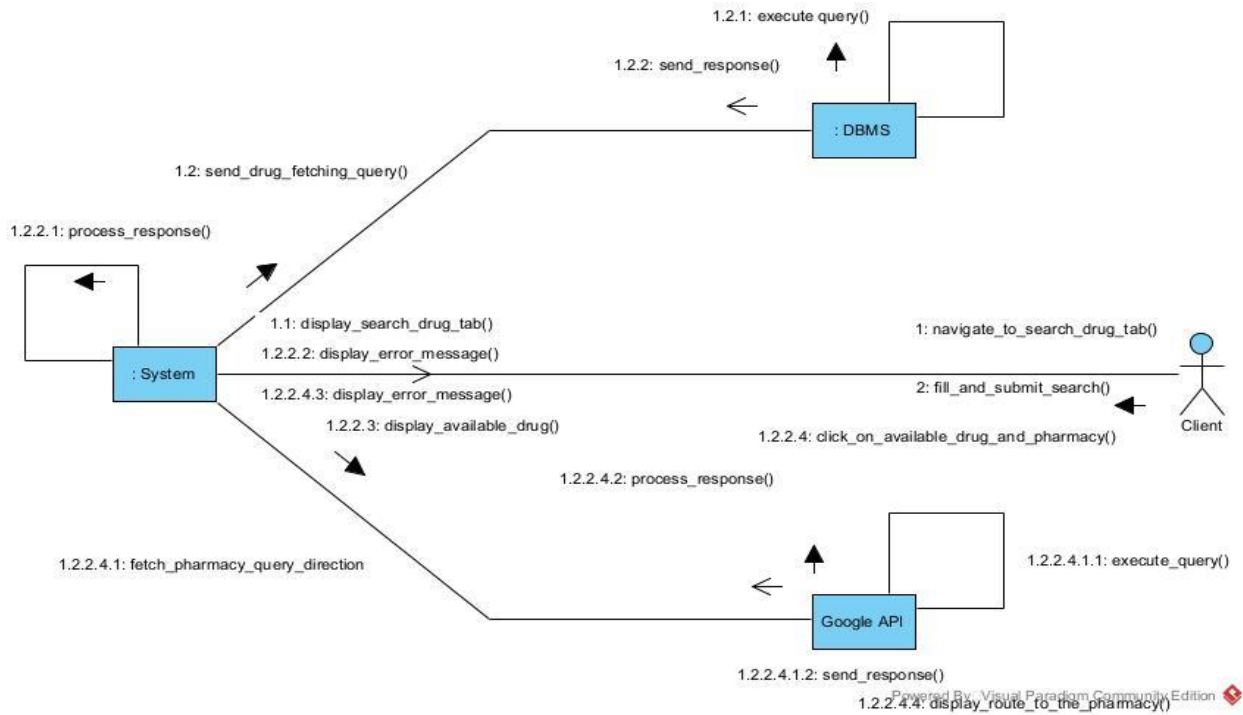
Element	Notation	Description
Message	2: message 2 →	Designs a particular communication between lifelines.
Actors		A role play by an entity that interacts with the subjects
Link		It initiates an association it connects two objects together for them to communicate.
Object		An actor represents an individual participant in the interaction conversation

d. Communication diagram of Authenticate



Figure 17: Authenticate Communication diagram

e. Communication diagram of Search Drug and Pharmacy



f. Communication diagram of Place Order

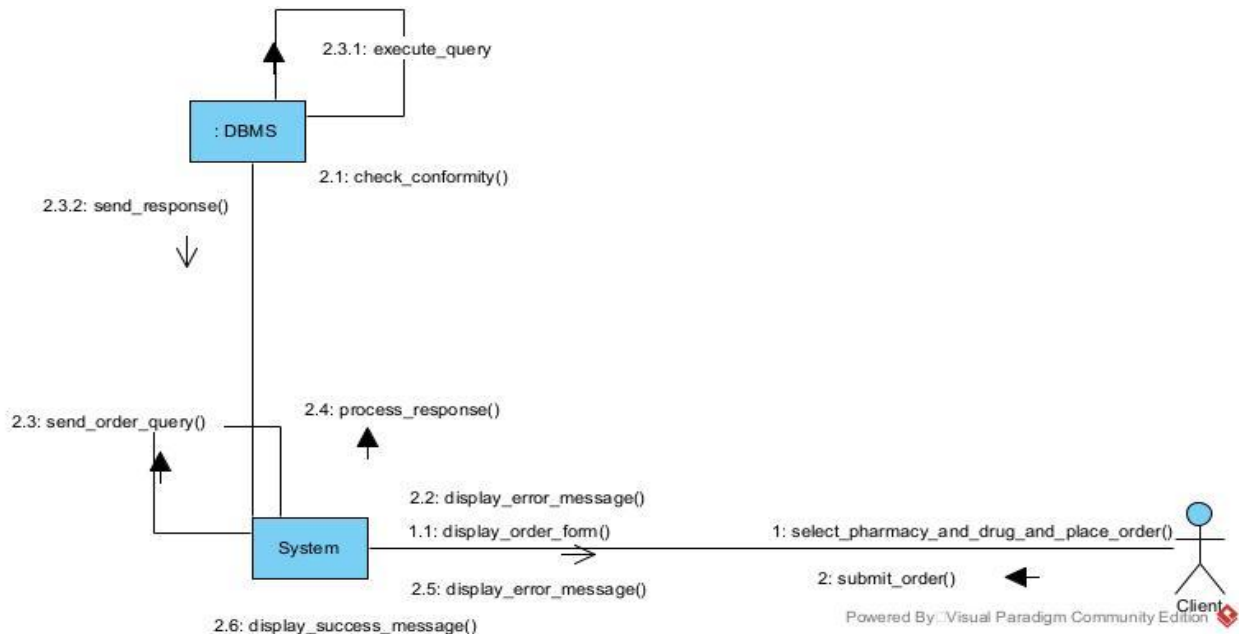


Figure 18: Communication diagram



5. STATE MACHINE DIAGRAM

a. Definition

A class diagram is a static diagram. It represents the static view of an application. class diagram is not only used for visualizing, describing and documenting different aspect of the system but also for constructing executable code of the software application. Class diagram describes the attribute and operation of a class and constraints imposed on the system. Its purpose is to model the static view of an application.

b. Formalism

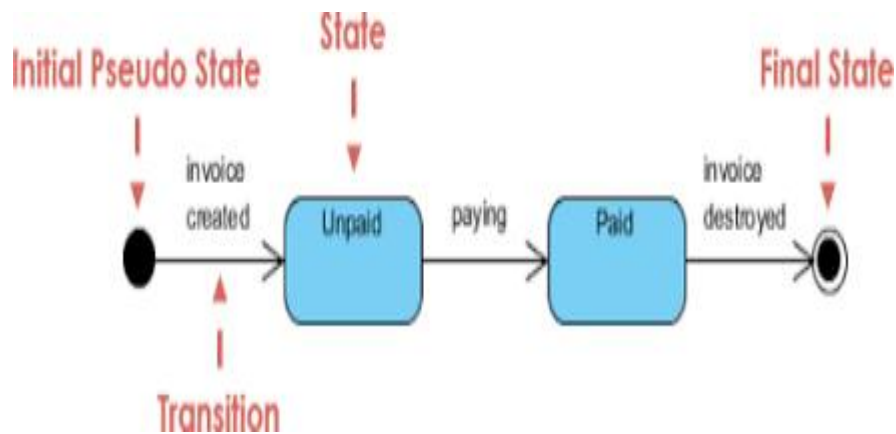
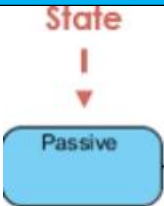
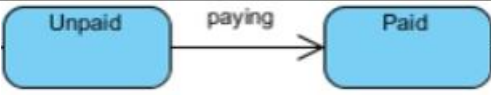
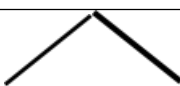


Figure 20: Formalism of a state machine diagram

b. Component of a state machine diagram

Table 22 : Component of state machine diagram

Element	Notation	Description
State		Models a situation during which a certain invariant condition holds.
First (Initial State)		It represents a default vertex, that is, a source for a single transaction to the default or composite state.
Final state		A state specifying that the enclosing region is complete
Transition		A direction relation between a source and a target vertex.
Choice pseudo state		A diamond symbol that indicates a dynamic condition with branched potential results
Terminate		Implies that the execution of a state by means of it context is terminated.
Diagram overview		A placeholder for the linked states in a state machine diagram.



d. State machine diagram of Account

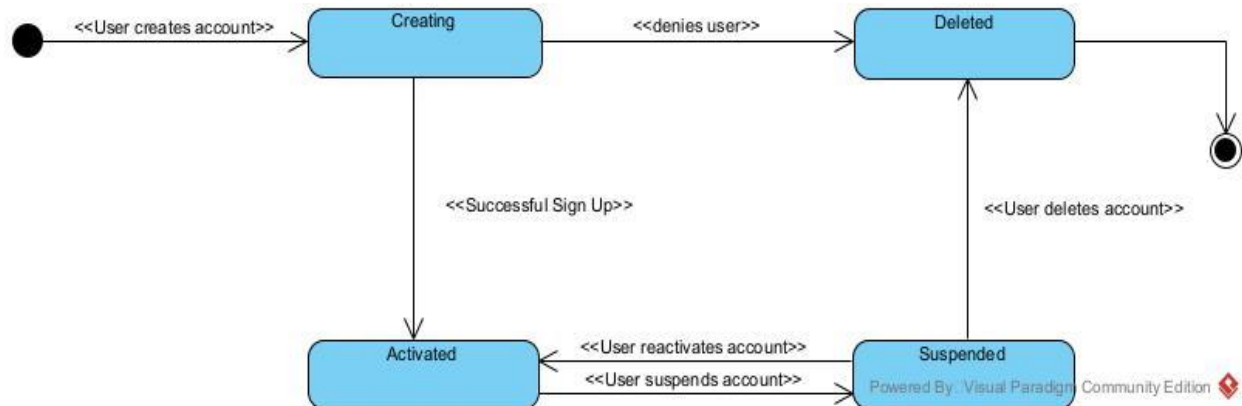


Figure 21: Account Creation State machine diagram

e. State machine diagram of Order

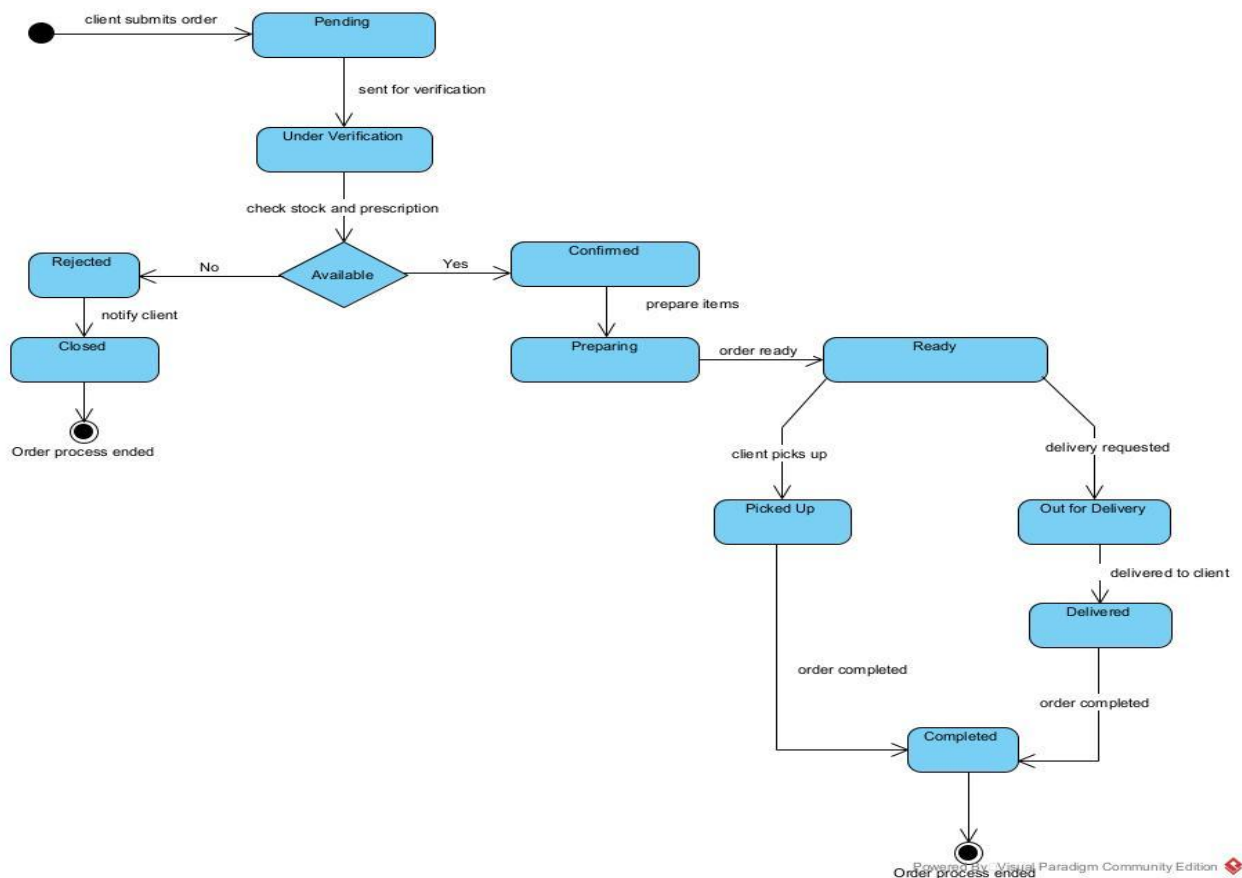


Figure 22: Order State machine diagram

Written By: NSAN SAMUEL PENIEL



CONCLUSION

During the analysis phase, we selected a software development process and modelling language. Next, we elaborated on the functional requirements of our system. We reviewed the use case diagram, which outlines the relationship between actors and the actions they can perform in the system, as well as the communication diagram that illustrates the architecture of the system based on object-oriented programming. We also analysed the sequence diagram, which delineates the flow of messages between elements in the system. Lastly, we reviewed the activity diagram, which displays the workflow of our system. We will proceed to the conception phase, during which we shall highlight the Technical branch of our system, accompanied by relevant diagrams.



FILE 4: CONCEPTION PHASE



Preamble

The conception phase is the part of the document that shows the link between the analysis and the realization phases. It is a continuation of the analysis phase, which represents the technical aspects used in modelling our system.

Content Overview

INTRODUCTION

I. TECHNICAL BRANCH

A. GENERIC DESIGN

B. CAPTURE OF TECHNICAL NEEDS

C. RELATED UML DIAGRAMS

i. CLASS DIAGRAM

ii. PACKAGE DIAGRAM

CONCLUSION



INTRODUCTION

The concept stage will outline the precise requirements, characteristics and activities needed to fulfil the operational needs of the suggested system, as defined during the analysis stage. The objective during this phase is to recognize and examine key components (either hardware or software), configuration (network capacity), processes and procedures, which are indispensable for the system to achieve its desired outcomes. We will scrutinize several diagrams such as the class diagram, state machine diagram and package diagram.



I. TECHNICAL BRANCH

A. GENERIC DESIGN

a. Hardware diagram of the system

The hardware diagram simply shows how the system components of our system are deployed; it shows the positioning of each device into its right proportion.

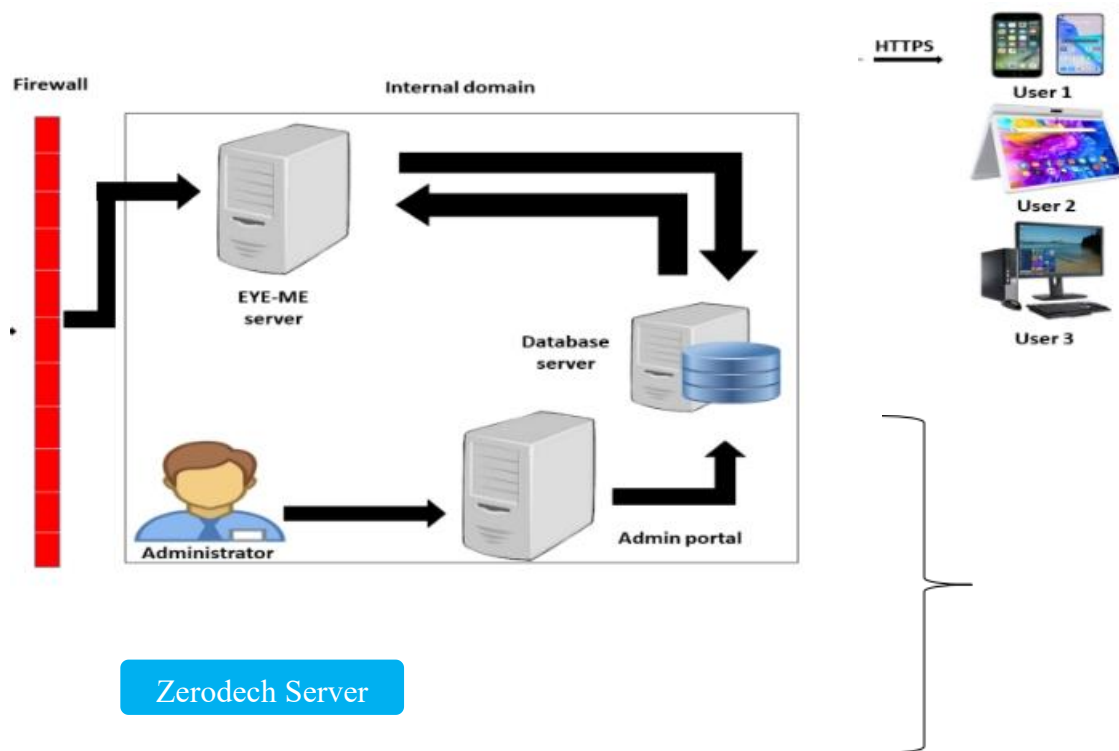


Figure 23 : Hardware diagram

b. High Level Architecture of the System

The high-level architecture diagram provides an overview of the entire system, identifying the main components that would be developed for the product and their interfaces.

B. CAPTURE OF TECHNICAL NEEDS

a. Physical Architecture

The design of the DBMS depends on its architecture. An n-tier architecture partitions overall system into related but separated n modules, which can be independently modified, altered, changed or replace. Within the scope of our project, we made use of the 3-tier architecture this architecture separate it tiers from each other based upon the user and the manipulated data in the database. Each layer has a well-defined communication interface, and the evolution of the layer is independent of the other. The 3-tier of our system is made up of:

- The client tier, which is also known as our presentation interphase.
- Application Tier, which represents our webserver.
- The data tier, which represents our DBMS server

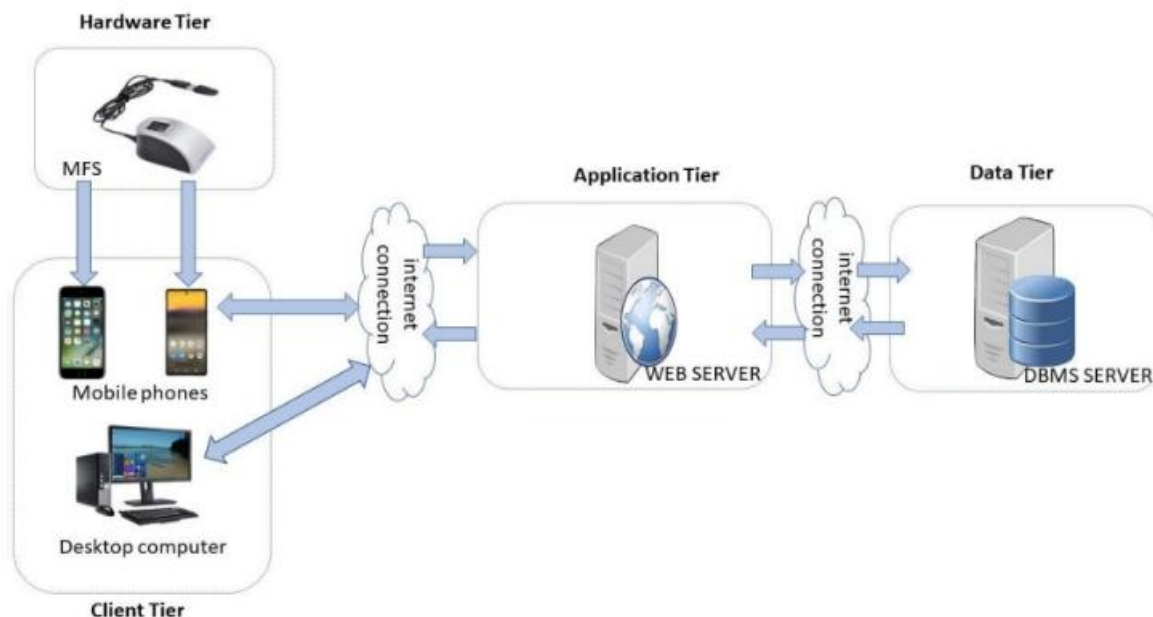


Figure 24 : n-tier Architecture (Source: <https://www.pinterest.es/pin/752241943987565648/>)



b. Logical Architecture

Model View controller or MVC as it is popularly called, is a software design pattern for developing application. A model view controller pattern is made up of the following three parts.

- ❖ **Model:** The lowest level of the pattern, which is responsible for maintaining data.
- ❖ **View:** This is responsible for displaying all or a portion of data to the user.
- ❖ **Controller:** It handles software codes that controls the interactions between the model and the view.

MVC is popular as it isolates the application logic from the user interface and supports separation of concerns. Here the controller receives all requests for the application then works with the model to prepare data needed by the view. The view then uses the data prepared by the controller to produce a final response. The MVC can be represented as follows:

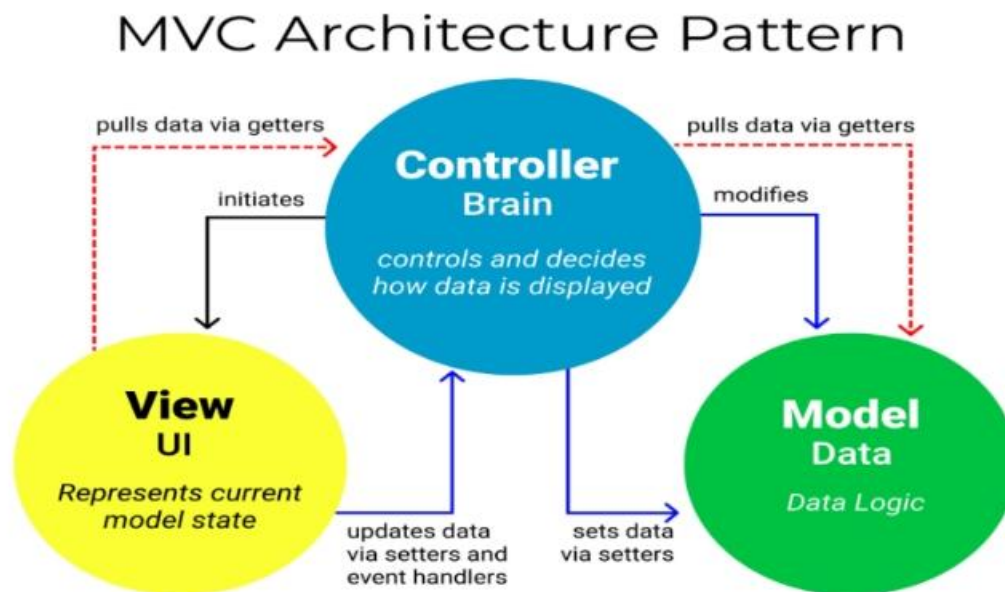


Figure 25 : The MVC architecture (source: <https://www.freecodecamp.org/news/the-model-view-controller-pattern-mvc-architecture-and-frameworks-explained/>)

C. RELATED UML DIAGRAMS

I. CLASS DIAGRAM

a. Definition

A class diagram is a static diagram. It represents the static view of an application. class diagram is not only used for visualizing, describing and documenting different aspect of the system but also for constructing executable code of the software application. Class diagram describes the attribute and operation of a class and constraints imposed on the system. Its purpose is to model the static view of an application.

b. Formalism

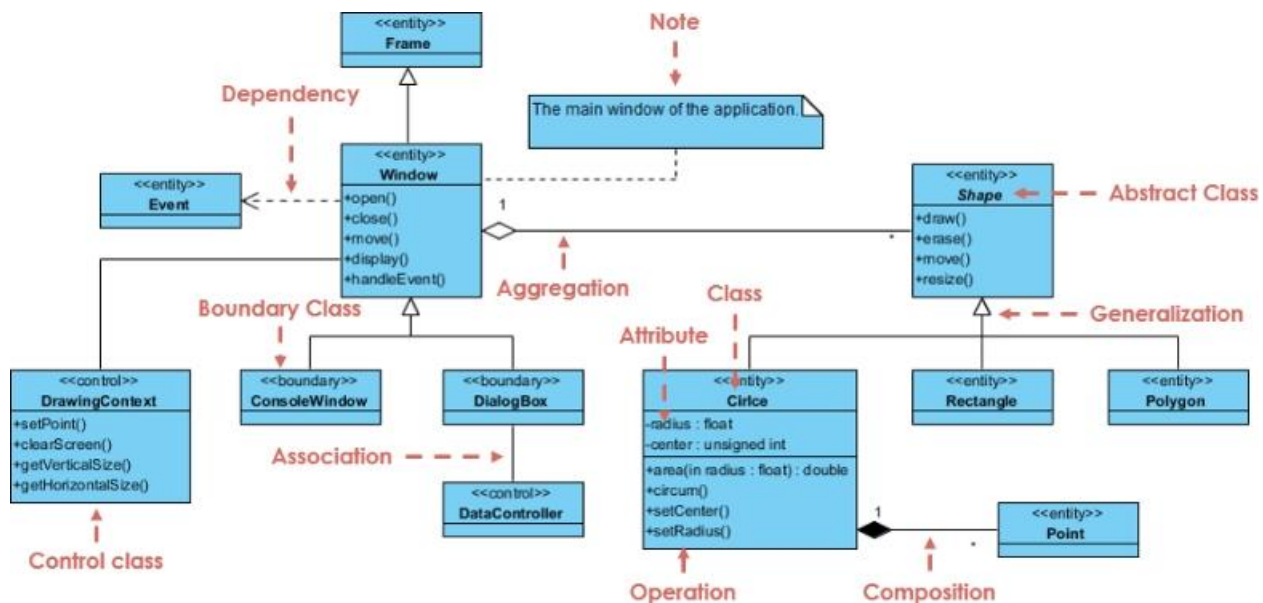
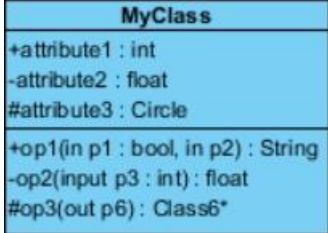
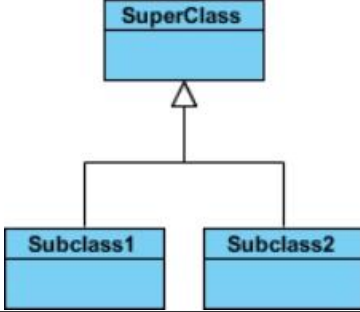
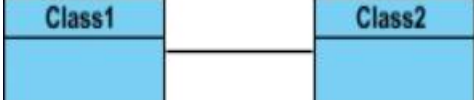
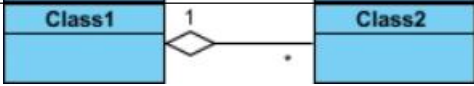
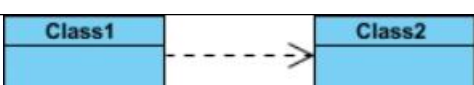


Figure 26 : Class diagram formalism



c. Component of a Class diagram

Table 23 : Components of class diagram

Element	Notation	Description
Class		A class is an element that defines the attributes and behaviors that an object can generate
Generalization		it a relationship between a whole thing (called superclass) and a more specific thing (called subclass)
Association		It is a general type of relationship between elements, it may include cardinality, roles etc.
Aggregation		If the parent of the aggregate is deleted, the children are not deleted.
Composition		If a parent of a composite is deleted all its parts are deleted with it.
Dependency		It existed between two classes, if one changes it may cause the change in the order, but the other way around.

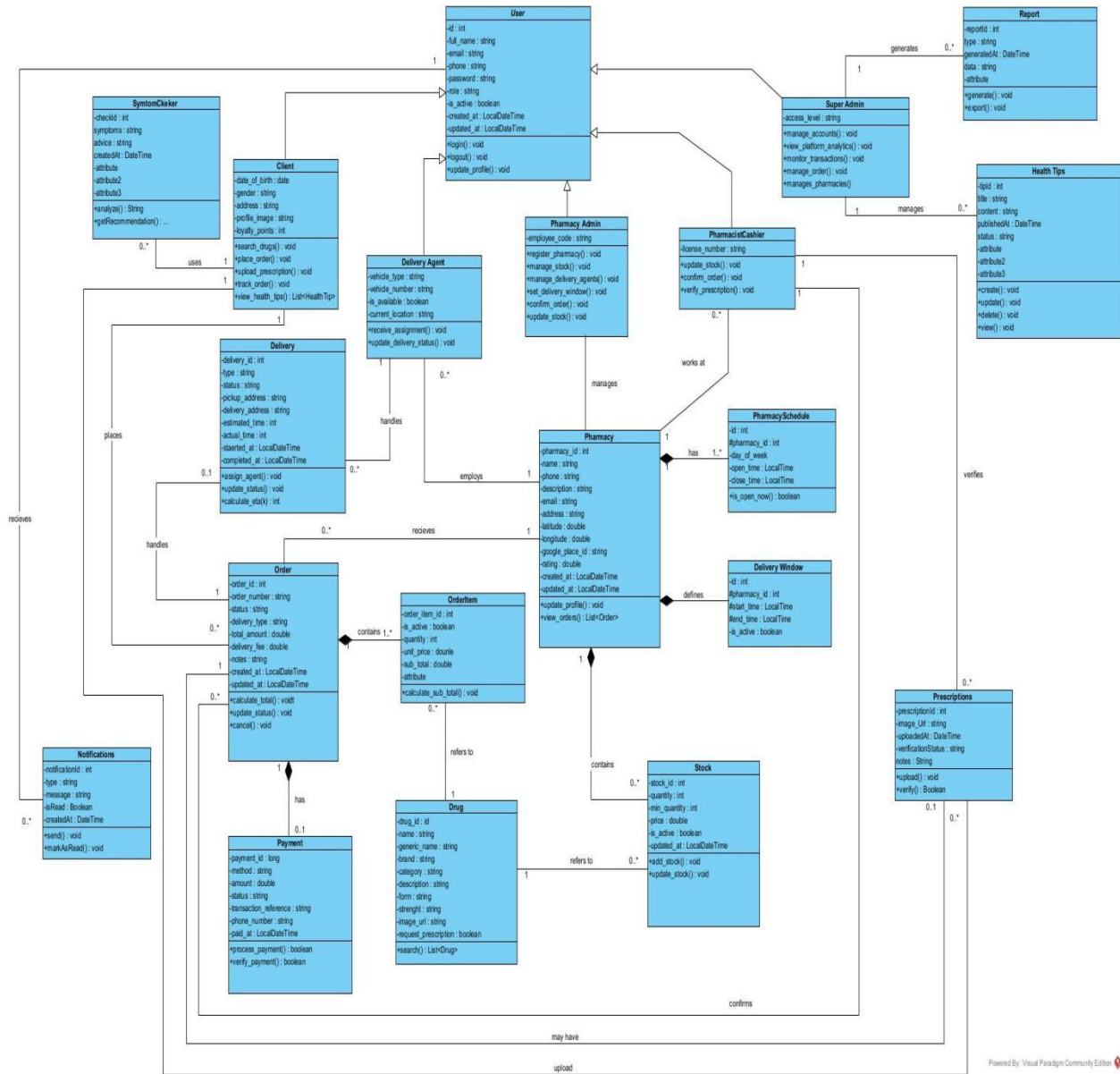


Figure 27 : Class diagram

2. PACKAGE DIAGRAM

a. Definition

This is a structural diagram used to show the organization and arrangement of various model elements in the form of packages.

b. Formalism

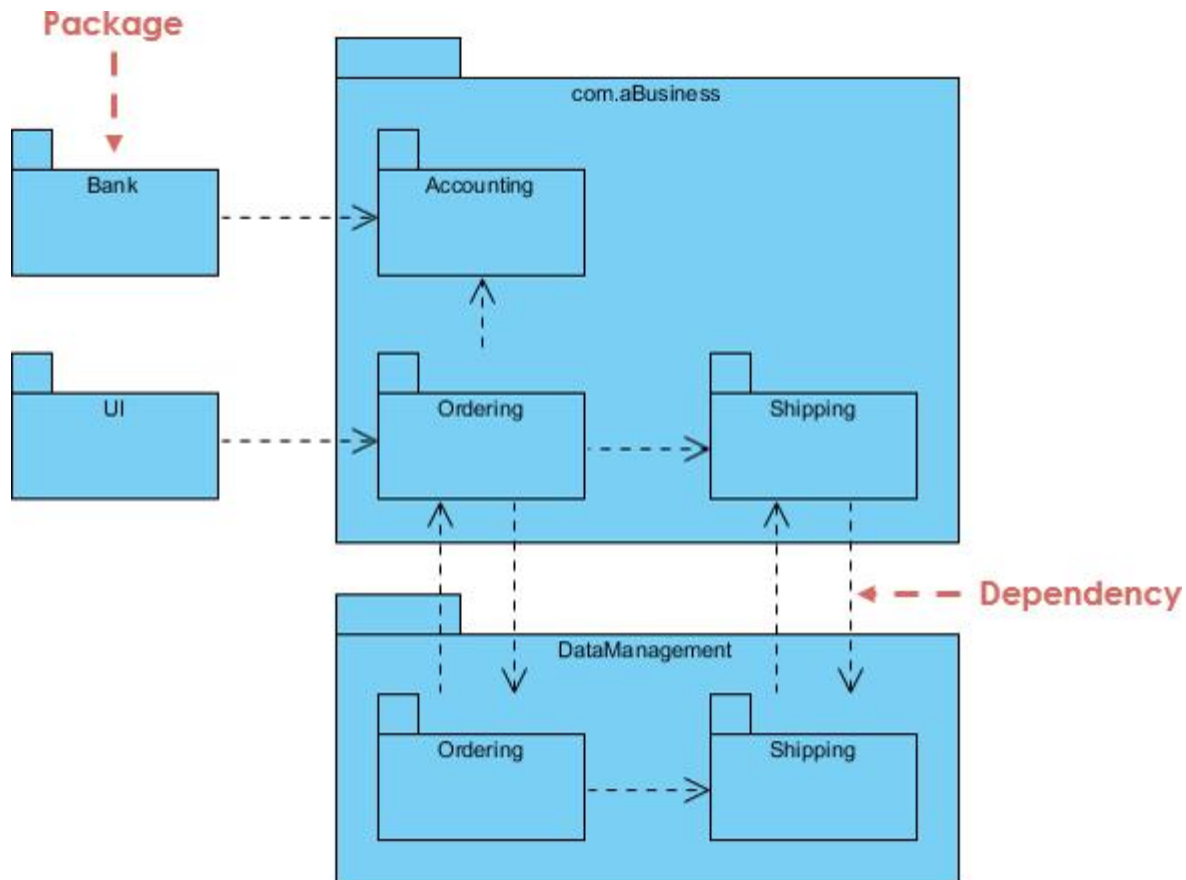
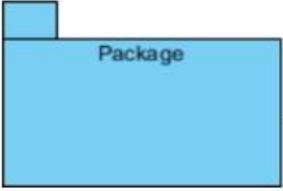
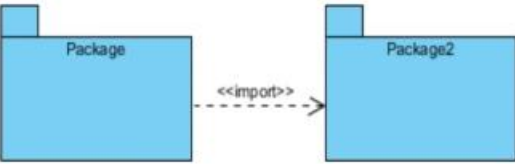
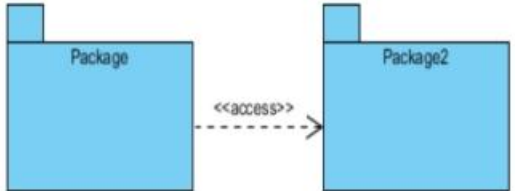


Figure 28 : Package diagram formalism

c. Component of package diagram

Table 24 : Component of package diagram



Element	Notation	Description
Package		A package is a namespace use to group related elements; it is a mechanism used to group elements into a better structure in a system.
Package import		A relationship Indicate that, functionality has been imported from one package to another.
Package access		A relationship Indicates that one package requires assistance from the function of another package.
Package merge		A relationship shows that, the functionality of two packages are combines to a single function.

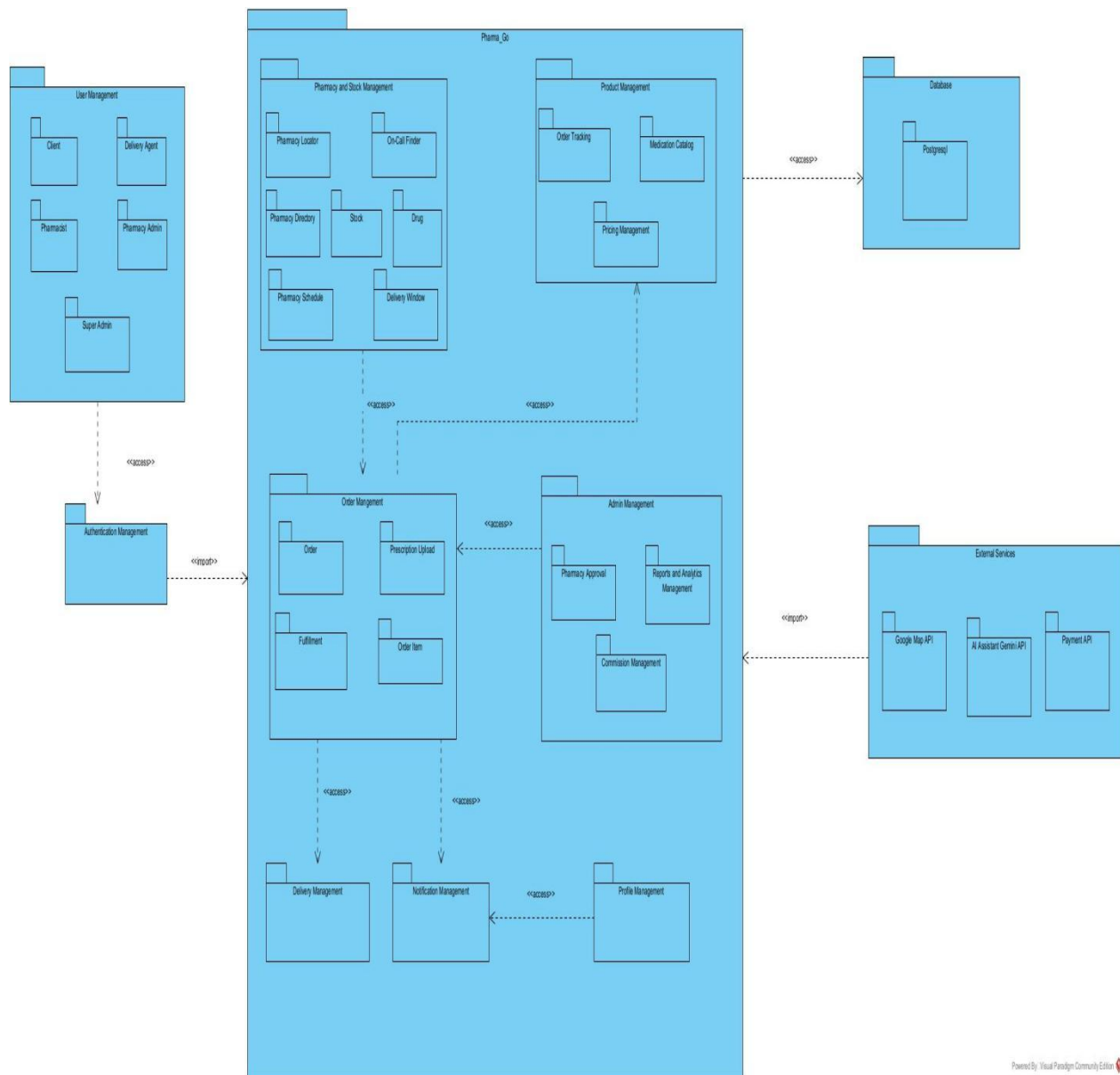


Figure 29 : package diagram



CONCLUSION

We presented the structure of our system and its integration into the existing technical architectures. In the initial stage, we aimed to investigate various aspects of our system. Primary actions included assessing the technical limitations, identifying the system's components with their classification and deployment onto relevant deployment targets. We then concluded this phase by evaluating the interactions between different components and participants of our system. Moving forward, our report will now progress to the implementation phase, which will entail concentrating on executing our system-related elements.



FILE 5: CONCEPTION PHASE



Preamble

In this phase we will go straightforward in the implementation of our solution, we will base ourselves on the analysis and conception phases, also present the component and deployment diagrams.

Content Overview

INTRODUCTION

I. COMPONENT DIAGRAM

II. DEPLOYMENT DIAGRAM

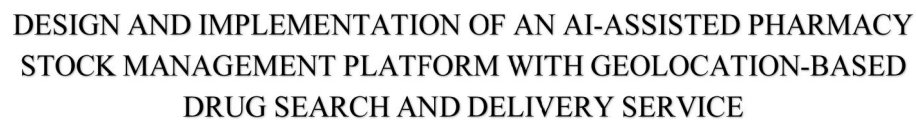
III. TECHNICAL CHOICES

CONCLUSION



INTRODUCTION

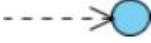
Here in the realization phase, we will concentrate on building or implementing our solution, we will review diagrams concerning the physical aspects of the system, such as libraries, documents and the physical topology of the system components when the software is deployed. Based on the various analyses and concepts developed earlier, this phase is as critical as the previous ones. It focuses on the physical topology of deployed components, the tools used to realize the application, the chosen programming language, and the overall architecture as it is implemented in code and infrastructure.





c. Component of Component diagram

Table 25 : Component of component diagram

Element	Notation	Description
Dependency		A dependency is a relationship that signifies that a single or a set of model elements requires other model elements for their specification or implementation.
Port		A port is often used to help expose required and provided interfaces of a component.
Interface		It is a point of interaction between a component and its environment or other components. It specifies the services that a component provides or requires from other components
Component		It represents a self-contained and replaceable unit that interacts with other components via provided and required interfaces

d. Component diagram

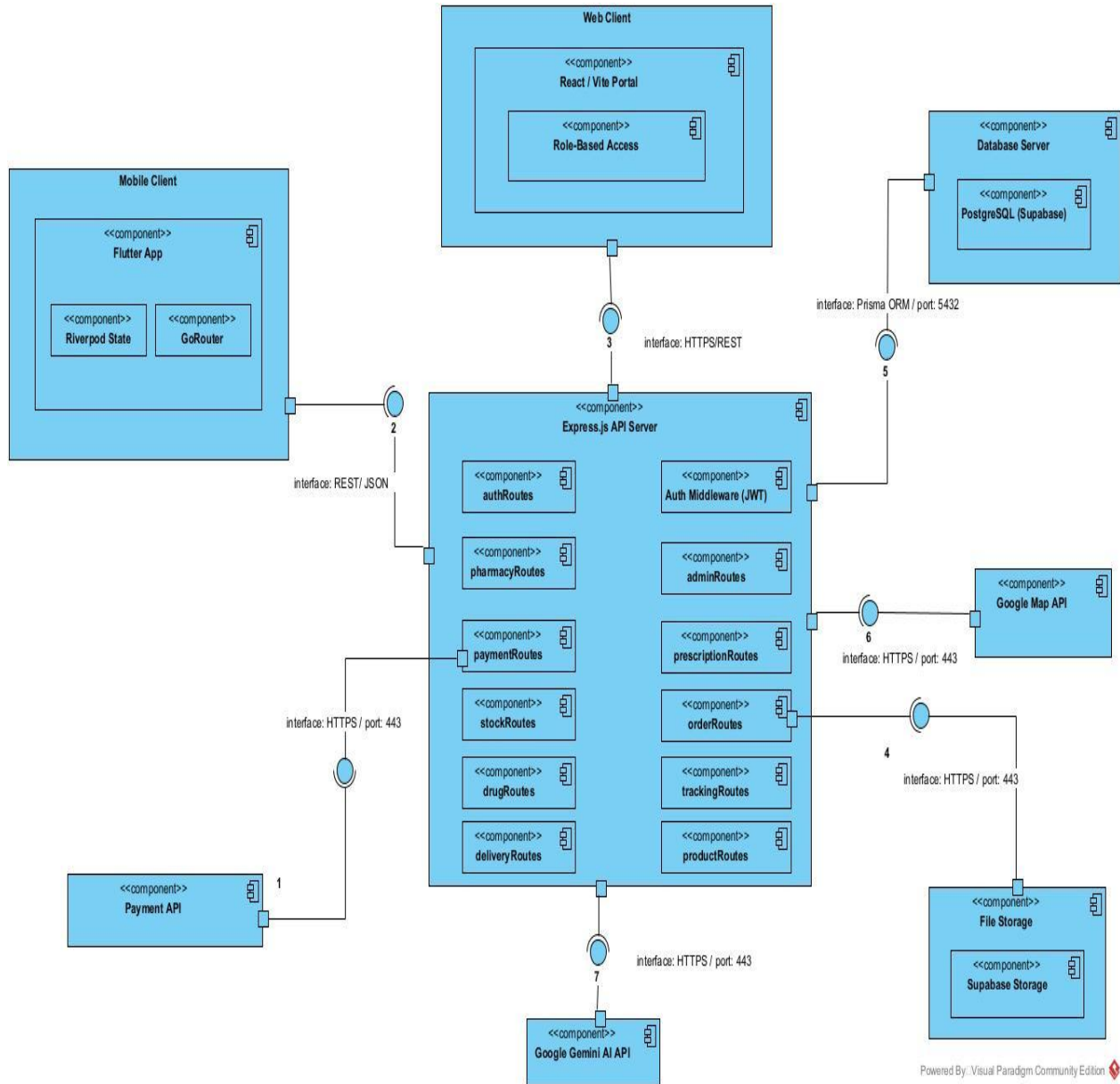
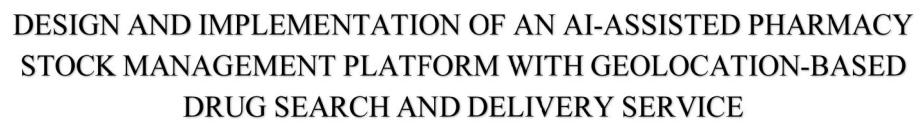


Figure 31 : component diagram



II. DEPLOYMENT DIAGRAM

a. Definition

A deployment diagram is a visual representation of the physical architecture of a computer system. The diagram represents the physical disposition of materials resources that constitute the system and shows the component repartition (software elements) that are executed inside these materials. It shows how the different hardware and software components of a system are connected and deployed on different nodes.

b. Formalism

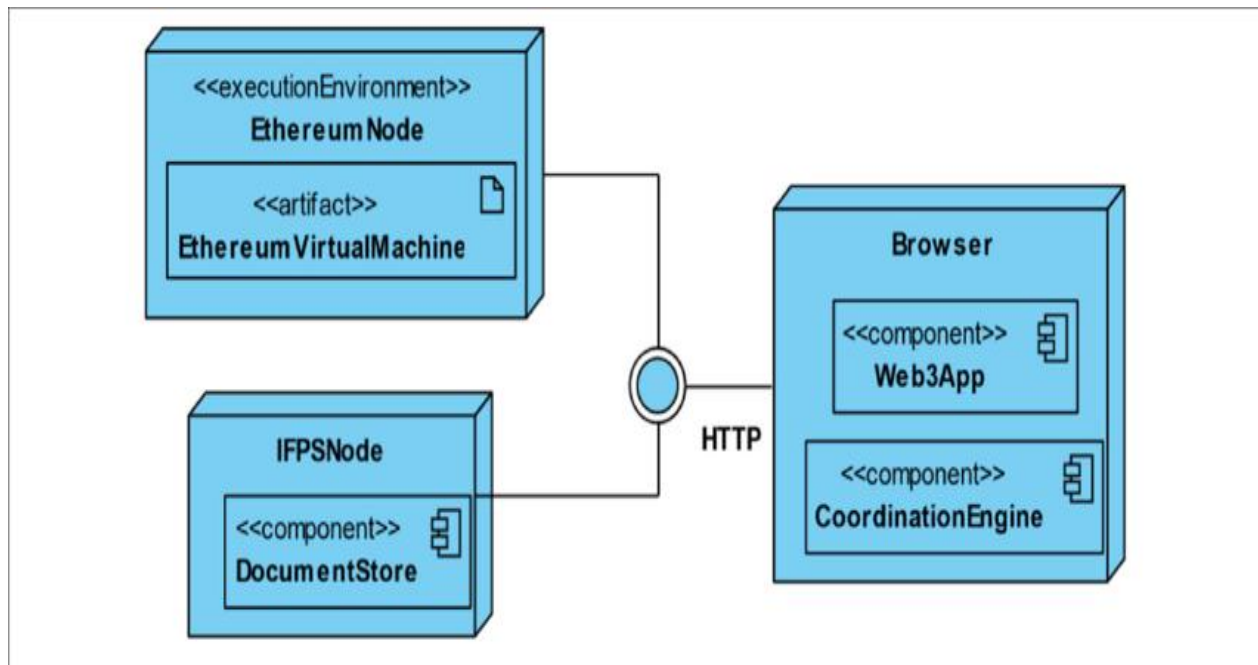
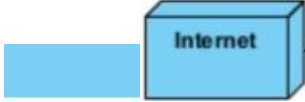
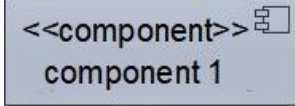


Figure 32 : Formalism of Deployment diagram



b. Component of Deployment diagram

Table 26 : Component of deployment diagram

Element	Notation	Description
Node		A hardware or software object, shown by a three-dimensional box
Artifact		An artifact is a major product, which is produced or used during the development of a software. E.g. diagrams, data models, setup scripts
Component		It represents a modular part of a system that encapsulates its content and whose manifestation is replaceable within its environment.
Association		An association helps to connect two nodes together which permits them to communicate together



c. Deployment diagram

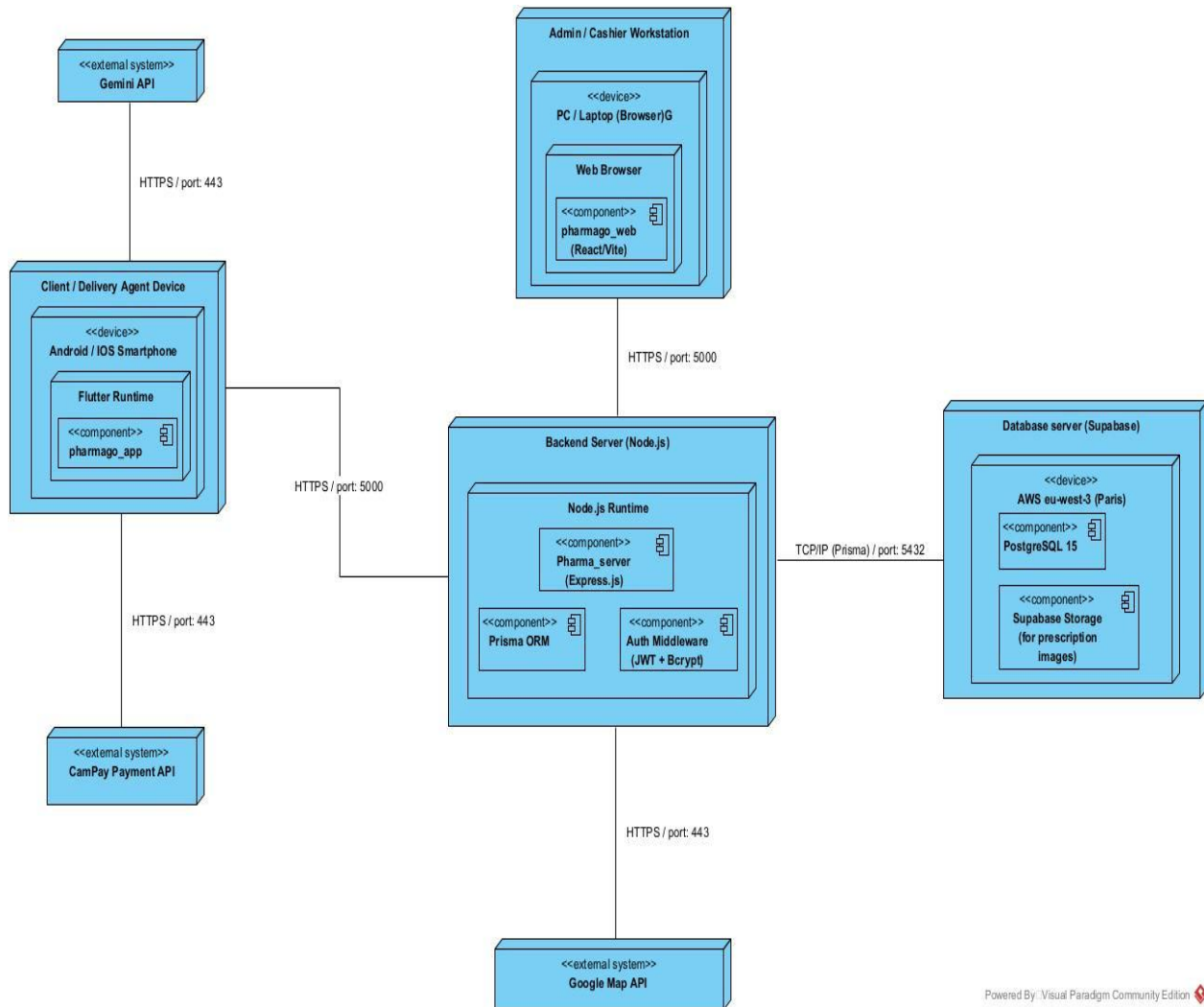


Figure 33 : deployment diagram



III. TECHNICAL CHOICES

To set up our platform and meet the needs of the end-users, the technologies chosen are React Native for the front-end, combined with Node.js for the back-end, and MongoDB for interaction with the database. Each of these technologies will be presented in detail in the following paragraphs.

a. Flutter

Flutter is an open-source UI framework developed by Google for building applications from a single codebase. It uses the Dart programming language and provides a rich collection of widgets for creating responsive and interactive user interfaces. In PharmaGo, Flutter is used to develop the **mobile application for clients**, providing features such as drug search, pharmacy location, order placement, prescription upload, payment, and order tracking.

b. React and Vite:

React is an open-source JavaScript library used to build interactive user interfaces, particularly for web applications. **Vite** is a modern development tool that provides a fast development environment and optimized build process for web applications. In PharmaGo, React and Vite are used to develop the **web portal** intended for Pharmacy Admins, Pharmacist/Cashiers, and the Super Admin.

Figure 35 : node js logo

c. Node.js and Express.js:

Node.js is an open-source, cross-platform JavaScript runtime environment that enables JavaScript to be executed on the server side. It is well suited for developing scalable web services and APIs. **Express.js** is a lightweight web framework for Node.js that simplifies the development of HTTP APIs and server-side applications. In PharmaGo, Node.js and Express.js are used to implement the **backend API**, which manages authentication, pharmacy and stock management, orders, prescriptions, payments, deliveries, and communication with external services.



Figure 36 : logo

d. PostgreSQL

PostgreSQL is an open-source, object-relational database management system known for its reliability, data integrity, and support for complex queries and relationships. In PharmaGo, PostgreSQL is used to store and manage structured information such as users, pharmacies, drugs, stock records, orders, prescriptions, payments, and deliveries.

e. Prisma ORM

Prisma is a modern Object-Relational Mapping (ORM) tool for Node.js and TypeScript applications. It provides a structured way for the application to interact with relational databases while simplifying database queries and data management. In PharmaGo, Prisma is used as the intermediary between the Express.js backend and the PostgreSQL database.

f. Supabase Storage

Supabase Storage is a cloud-based storage service designed for storing and managing files. In PharmaGo, it is used to store **prescription images and other required documents** uploaded by clients. The backend controls access to these files according to the application's authorization rules.

e. Gantt Project;

Project Gantt is a widely used project management tool for planning and tracking project tasks over a given period. A Gantt chart is particularly useful for visualizing dependencies between tasks, start and finish dates, and project progress.



DESIGN AND IMPLEMENTATION OF AN AI-ASSISTED PHARMACY STOCK MANAGEMENT PLATFORM WITH GEOLOCATION-BASED DRUG SEARCH AND DELIVERY SERVICE



Figure 37 : Gantt Project logo

f. Visual paradigm;

Visual Paradigm Community Edition is a UML software that supports all UML diagram types. It is an international award-winning UML modeler, and yet it is easy-to-use, intuitive & completely free.



Figure 38 : Visual Paradigm

g. Postman

Postman is a popular API development tool that facilitates the creation, testing and documentation of APIs. It is widely used by developers, testers and API development teams to simplify the process of working with HTTP APIs.



Figure 39 : Postman logo

Written By: NSAN SAMUEL PENIEL



CONCLUSION

During our realization phase, we successfully developed and implemented our application. To achieve this, we utilized our analysis and conception phase. We also created deployment and component diagrams that illustrate the modules, files, and assets, as well as how different elements interact with each other within our system. Our next step is to move onto the functionalities testing phase, where we will assess the efficacy of the various modules present in our app for different users.



FILE 6: TEST OF FUNCTIONALITIES



Preamble

In this report, we will explore the different aspects of the test of functionalities and its significance in the software development life cycle.

Content Overview

INTRODUCTION

I. TEST OF FUNCTIONALITIES

II. TEST SHOWCASES

CONCLUSION



INTRODUCTION

The test of functionalities plays a crucial role in ensuring the quality and reliability of software systems. By thoroughly examining each functionality and validating its behavior, developers can identify potential issues early in the development process. This helps in reducing the overall cost and effort required for bug fixing and maintenance.



I. TEST OF FUNCTIONALITIES

1. What is Unit Testing

Unit testing involves testing the smallest parts of an application, such as functions or methods, independently from the rest of the application. The primary goal is to validate that each unit of the software performs as expected.

2. Unit Testing Importance

Unit testing ensures that each component of the application works independently and correctly. It helps catch bugs early in the development process and guarantees that future changes do not break existing functionality.

3. Testing Strategy

We implemented **unit testing** to isolate and test individual parts of the app, specifically the API endpoints and backend services. Unit testing provides fast feedback on code quality and behavior.

4. Tools Used

- **Jest:** testing framework used for running unit tests.
- **Supertest:** For HTTP testing for Express APIs requests.
- **MongoDB Server:** Used in some instances to mock MongoDB models for tests.

5. Key Functionalities Tested

a. User Registration

- Tests ensure that users can successfully register with valid details.
- Handles test cases where invalid inputs or missing fields cause registration failure.

b. User Authentication

- Tests confirm that users can log in with correct credentials.



- Includes scenarios where login fails due to invalid credentials (e.g., wrong password).

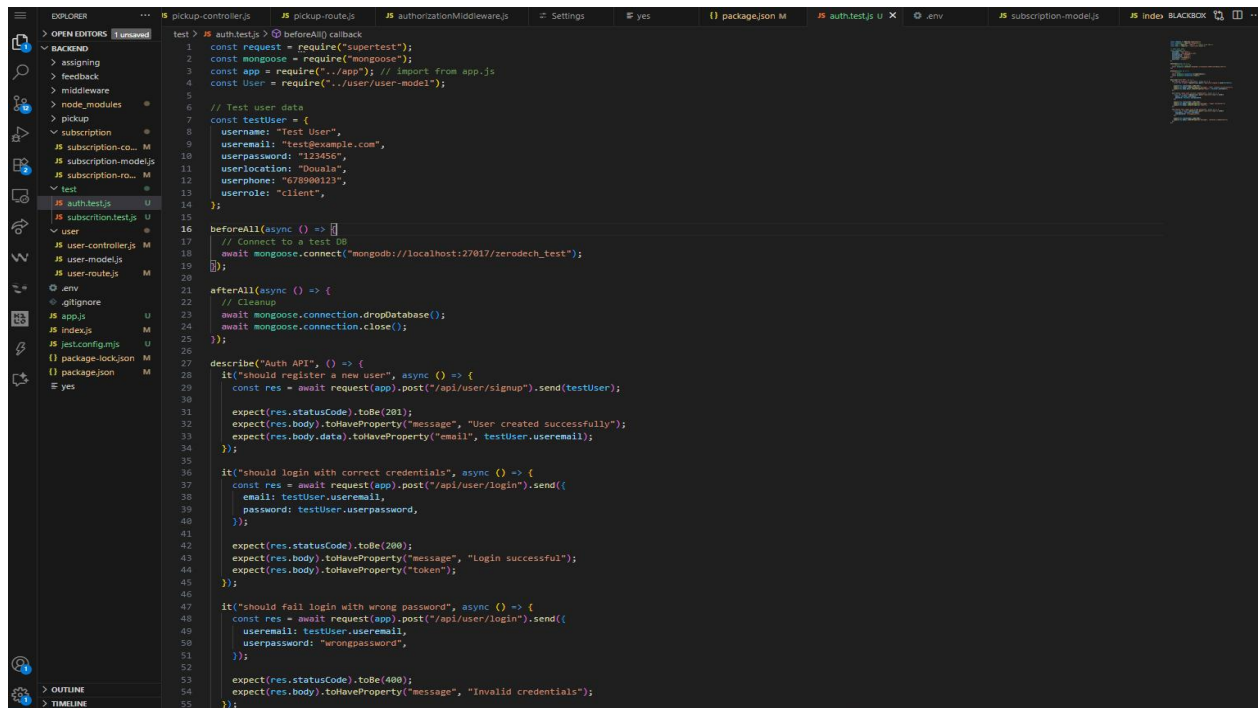
c. Subscription Creation

- Tests verify that subscription are created successfully with all required information.
- Includes test cases where an error is returned if a user tries to make a duplicate subscription for the same provider.

II. TEST SHOWCASES

a. Passing Unit Tests

- User Registration: Demonstrates successful user registration with valid data and shows successful login and token generation when valid credentials are provided.



```

test > JS pickup-controller.js JS pickup-route.js JS authorizationMiddleware.js Settings yes package.json M JS auth-test.js u X .env JS subscription-model.js JS index BLACKBOX
EXPLORER OPEN EDITORS
> BACKEND
  > assigning
  > feedback
  > middleware
  > node_modules
  > pickup
  > subscription
    JS subscription-co... M
    JS subscription-model.js
    JS subscription-ro... M
    JS test
      JS auth-test.js U
      JS subscription-test.js U
      JS user... M
      JS user-controller.js M
      JS user-model.js
      JS user-route.js M
    .env
    .gitignore
    JS app.js
    JS index.js
    JS jest.config.mjs U
    () package-lock.json M
    () package.json M
    yes
    OUTLINE
    TIMELINE

test > JS pickup-controller.js JS pickup-route.js JS authorizationMiddleware.js Settings yes package.json M JS auth-test.js u X .env JS subscription-model.js JS index BLACKBOX
1  const request = require("supertest");
2  const mongoose = require("mongoose");
3  const app = require("../app"); // Import from app.js
4  const User = require("../user/user-model");
5
6  // Test user data
7  const testUser = {
8    username: "Test User",
9    email: "test@example.com",
10   userpassword: "123456",
11   userlocation: "Dumaila",
12   userphone: "678900123",
13   userrole: "client",
14 };
15
16 beforeAll(async () => {
17   // Connect to a test DB
18   await mongoose.connect("mongodb://localhost:27017/zerodech_test");
19 });
20
21 afterAll(async () => {
22   // Cleanup
23   await mongoose.connection.dropDatabase();
24   await mongoose.connection.close();
25 });
26
27 describe("Auth API", () => {
28   it("should register a new user", async () => {
29     const res = await request(app).post("/api/user/signup").send(testUser);
30
31     expect(res.statusCode).toBe(201);
32     expect(res.body).toHaveProperty("message", "User created successfully");
33     expect(res.body.data).toHaveProperty("email", testUser.email);
34   });
35
36   it("should login with correct credentials", async () => {
37     const res = await request(app).post("/api/user/login").send({
38       email: testUser.email,
39       password: testUser.userpassword,
40     });
41
42     expect(res.statusCode).toBe(200);
43     expect(res.body).toHaveProperty("message", "Login successful");
44     expect(res.body).toHaveProperty("token");
45   });
46
47   it("should fail login with wrong password", async () => {
48     const res = await request(app).post("/api/user/login").send({
49       email: testUser.email,
50       userpassword: "wrongpassword",
51     });
52
53     expect(res.statusCode).toBe(400);
54     expect(res.body).toHaveProperty("message", "Invalid credentials");
55   });
56

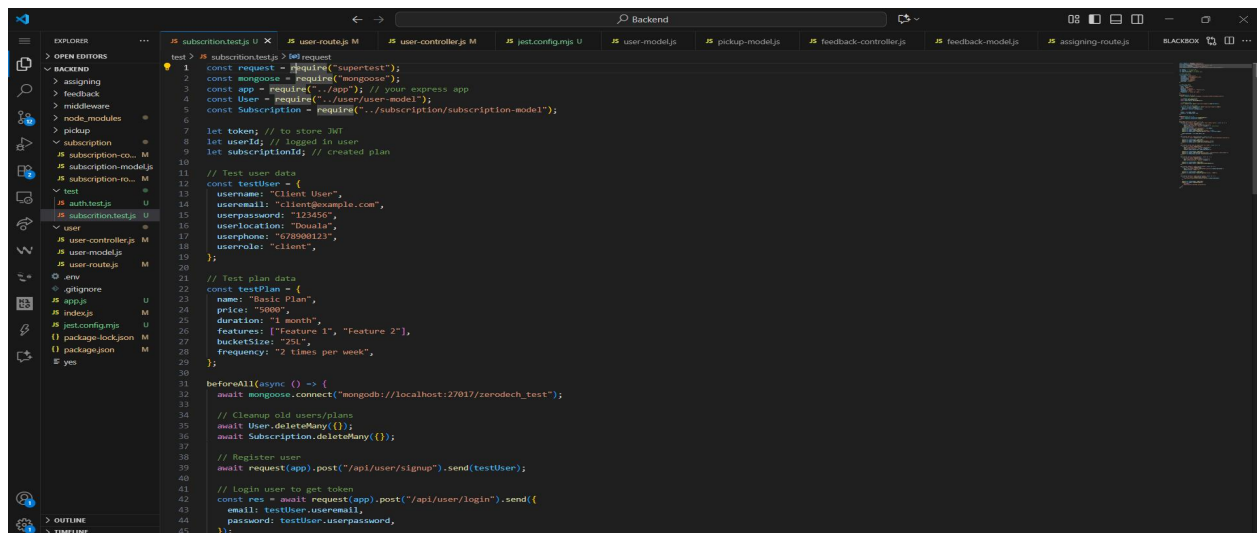
```

Figure 40 : User registration and login code test

```
Test Suites: 2 passed, 2 total
Tests: 9 passed, 9 total
Snapshots: 0 total
Time: 6.773 s, estimated 7 s
Ran all test suites.
Test Suites: 2 passed, 2 total
Tests: 9 passed, 9 total
Snapshots: 0 total
Time: 6.773 s, estimated 7 s
Ran all test suites.
Test Suites: 2 passed, 2 total
Tests: 9 passed, 9 total
Snapshots: 0 total
Time: 6.773 s, estimated 7 s
Ran all test suites.
```

Figure 41 : User registration and login test

- Client Place an order: Demonstrates successful subscribe client with valid data and shows successful subscription and token generation when valid credentials are provided.



```
test('clientPlaceOrder', () => {
  const request = require('supertest');
  const mongoose = require('mongoose');
  const app = require('../app'); // your express app
  const User = require('../user/user-model');
  const Subscription = require('../subscription/subscription-model');

  let token; // to store JWT
  let userId; // logged in user
  let subscriptionId; // created plan

  // Test user data
  const testUser = {
    username: 'Client User',
    userEmail: 'client@example.com',
    userpassword: '123456',
    userlocation: 'Douala',
    userphone: '978900123',
    userrole: 'client',
  };

  // Test plan data
  const testPlan = {
    name: 'Basic Plan',
    price: '5000',
    duration: '1 month',
    features: ['Feature 1', 'Feature 2'],
    bucketSize: '25L',
    frequency: '2 times per week',
  };

  beforeAll(async () => {
    await mongoose.connect('mongodb://localhost:27017/zerotech_test');

    // Cleanup old users/plans
    await User.deleteMany({});
    await Subscription.deleteMany({});

    // Register user
    await request(app).post('/api/user/signup').send(testUser);

    // Login user to get token
    const res = await request(app).post('/api/user/login').send({
      email: testUser.useremail,
      password: testUser.userpassword,
    });
  });
```

Figure 42 : Client Place Order test

```
Test Suites: 2 passed, 2 total
Tests: 9 passed, 9 total
Snapshots: 0 total
Time: 6.773 s, estimated 7 s
Ran all test suites.
Test Suites: 2 passed, 2 total
Tests: 9 passed, 9 total
Snapshots: 0 total
Time: 6.773 s, estimated 7 s
Ran all test suites.
Test Suites: 2 passed, 2 total
Tests: 9 passed, 9 total
Snapshots: 0 total
Time: 6.773 s, estimated 7 s
Ran all test suites.
```

Figure 43 : Client Subscription test



CONCLUSION

Testing is an integral part of the software development life cycle and is performed to verify the correctness, reliability, and robustness of the implemented functionality. The functionality and testing phase is therefore a critical stage in software development where the implemented features are validated, and any issues are identified and resolved.



FILE 7: INSTALLATION GUIDE AND USER GUIDE



Preamble

A user guide explains how to use a software application in a language that a non-technical person can understand. Thus, it enables the user to easily use the application to familiarize themselves with the software and discover all its functionalities.

Content Overview

INTRODUCTION

- I. DESCRIPTION OF THE APPLICATION
- II. USER MANUAL

CONCLUSION



INTRODUCTION

The user manual is the document established after the realization of an application, software, or platform. It informs on the question "how to use the software, the application, or the platform that you have in front of you? ". It is therefore crucial for us to offer the various users of our platform a guide allowing them to easily perform various operations on our platform.



I. DESCRIPTION OF THE APPLICATION

Zerodech is a waste management application developed to address the growing problem of uncollected waste in Cameroon, which contributes to pollution, airborne diseases, and poor urban sanitation. The app provides a digital solution that connects households, waste collectors, and administrators to ensure efficient and transparent waste collection.

The system works in **three layers**:

- **Customers (Households/Businesses):** Subscribe to waste collection plans, make payments, and track pickups.
- **Collectors (Field Workers):** Receive assigned pickups, navigate routes via GPS, and confirm collection with photo proof.
- **Administrators (Supervisors/City Councils):** Monitor operations, assign collectors, track payments, and generate reports.

II. USER MANUAL

1. PREREQUISITES



CONCLUSION

This user guide has provided a comprehensive overview of our product's features, functionalities, and best practices for optimal use. By following the instructions and tips outlined in this document, you'll be well-equipped to maximize the potential of our product and streamline your workflow.



PERSPECTIVES

III. FUTURE PERSPECTIVES

Having developed the core functionalities of **PharmaGo**, we plan to further improve the platform with the following features:

- ✧ **Enhanced Mobile Money payment integration** to provide secure and convenient payment processing for medicine orders and delivery services.
- ✧ **Advanced user profile management** to allow clients to manage their personal information, preferences, addresses, and order history.
- ✧ **Health consultation and counseling services** to facilitate communication between clients and qualified healthcare professionals when necessary.
- ✧ **Daily health tips and guidance** to provide users with useful health information while maintaining the advisory nature of the platform.
- ✧ **Enhanced security and user verification mechanisms** to strengthen account protection, verify pharmacy and healthcare-related users, and improve the overall reliability of the platform.
- ✧ **Improved AI-assisted services** to enhance the symptom-checking feature while maintaining appropriate limitations and encouraging users to consult qualified healthcare professionals when required.



GENERAL CONCLUSION

GENERAL CONCLUSION

The project focused on the **design and implementation of PharmaGo, an AI-assisted pharmacy stock management platform with geolocation-based drug search and delivery service**. The project successfully resulted in the development of a digital solution designed to improve the visibility and management of pharmacy stocks while facilitating access to medicines for clients. Through features such as pharmacy and stock management, geolocation-based drug search, order processing, prescription management, pickup and delivery services, Mobile Money payment, and order tracking, PharmaGo provides a centralized platform connecting clients with pharmacies and supporting pharmacy personnel in managing their daily operations. The development process involved the analysis of user requirements, feasibility studies, UML modeling, system design, implementation, and testing following the **2TUP process**. The project also required the integration of several technologies and external services, including **Flutter, React/Vite, Node.js, Express.js, PostgreSQL, Prisma, Google Maps API, Gemini API, Supabase Storage, and Mobile Money services**. Particular attention was given to role-based access control, database management, API integration, prescription handling, and the coordination of pickup and delivery services. The implementation of PharmaGo provided valuable practical experience in **software engineering, mobile and web application development, database management, API integration, geolocation services, artificial intelligence, system security, UML modeling, and project management**. It also strengthened our ability to analyze real-world problems and translate them into functional and technically coherent software solutions.

Written By: NSAN SAMUEL PENIEL



DESIGN AND IMPLEMENTATION OF AN AI-ASSISTED PHARMACY
STOCK MANAGEMENT PLATFORM WITH GEOLOCATION-BASED
DRUG SEARCH AND DELIVERY SERVICE



ANNEXES

Written By: NSAN SAMUEL PENIEL

113



BIBLIOGRAPHY

- Mercurial 2022 – 2025
- UML @ Classroom an introduction Object – Oriented Modelling, Gerti Kappel, Martina Seidl 2015.
- [React Native for Mobile Development: Harness the Power of React Native to Create Stunning IOS and Android Applications](#) 2nd edition Abhishek Nalwaya, 2019

Written By: NSAN SAMUEL PENIEL

I



WEBOGRAPHY

- <https://www.geeksforgeeks.org/how-to-install-mongodb-on-windows/> Here we got the steps for installation of mongodb.
- <https://stackoverflow.com/questions/78131028/country-picker-in-react-native> Here we fine our country picker.
- <https://www.meetup.com/about/> We got information about our existing system meetup.
- https://www.youtube.com/watch?v=TIB_eWDSMt4&list=PPSV We got a full course under node js.
- <https://reactnative.dev/> we got here a course on how to use react native.
- <https://expo.dev/go> we got stips on how to install and use expo go for our project
- <https://www.freecodecamp.org/news/the-model-view-controller-pattern-mvc-architecture-and-frameworks-explained/> Here we got the MVC diagram.
- <https://www.mysciencework.com/omniscience/pervasive-mobile-healthcare-system-based> here got the 2TUP diagram
- <https://redux.js.org/tutorials/essentials/part-5-async-logic> Async Logic and Data Fetching



DESIGN AND IMPLEMENTATION OF AN AI-ASSISTED PHARMACY
STOCK MANAGEMENT PLATFORM WITH GEOLOCATION-BASED
DRUG SEARCH AND DELIVERY SERVICE



TABLE OF CONTENTS

Written By: NSAN SAMUEL PENIEL

III